

Отчёт для третьего модуля внешнего курса

Модуль 3

Семенченко Татьяна Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение курса	7
3.1	Текстовый редактор vim	7
3.2	Скрипты на bash: основы	11
3.3	Скрипты на bash: ветвления и циклы	15
3.4	Скрипты на bash: разное	20
3.5	Продвинутый поиск и редактирование	28
3.6	Строим графики в gnuplot	34
3.7	Разное	38
4	Выводы	44

Список иллюстраций

3.1		8
3.2		9
3.3		10
3.4		11
3.5		12
3.6		13
3.7		14
3.8		15
3.9		16
3.10		17
3.11		18
3.12		19
3.13		20
3.14		21
3.15		22
3.16		23
3.17		24
3.18		25
3.19		27
3.20		28
3.21		29
3.22		30
3.23		31
3.24		32
3.25		33
3.26		34
3.27		35
3.28		36
3.29		37
3.30		38
3.31		39
3.32		40
3.33		41
3.34		42
3.35		43

Список таблиц

1 Цель работы

Пройти внешний курс «Введение в Linux!»

2 Задание

Прохождение 3 модуля внешнего курса на stepic «Введение в Linux!».

3 Выполнение курса

3.1 Текстовый редактор vim

” : «, затем»q”, затем «Enter» - в vim мы по умолчанию находимся в нормальном режиме, но чтобы выполнить команду выхода, нужно перейти в командный режим с помощью «:», затем ввести «q» и нажать Enter. Просто q или Q не сработают, так как они либо ничего не делают в нормальном режиме, либо переключают режим. (?@fig-01)

!()image/3.1.png

Суть задания: word — буквы, цифры, _; WORD — любые непробельные символы. Из-за разного состава символов W перемещается быстрее и останавливается только на пробелах (рис. 3.1).

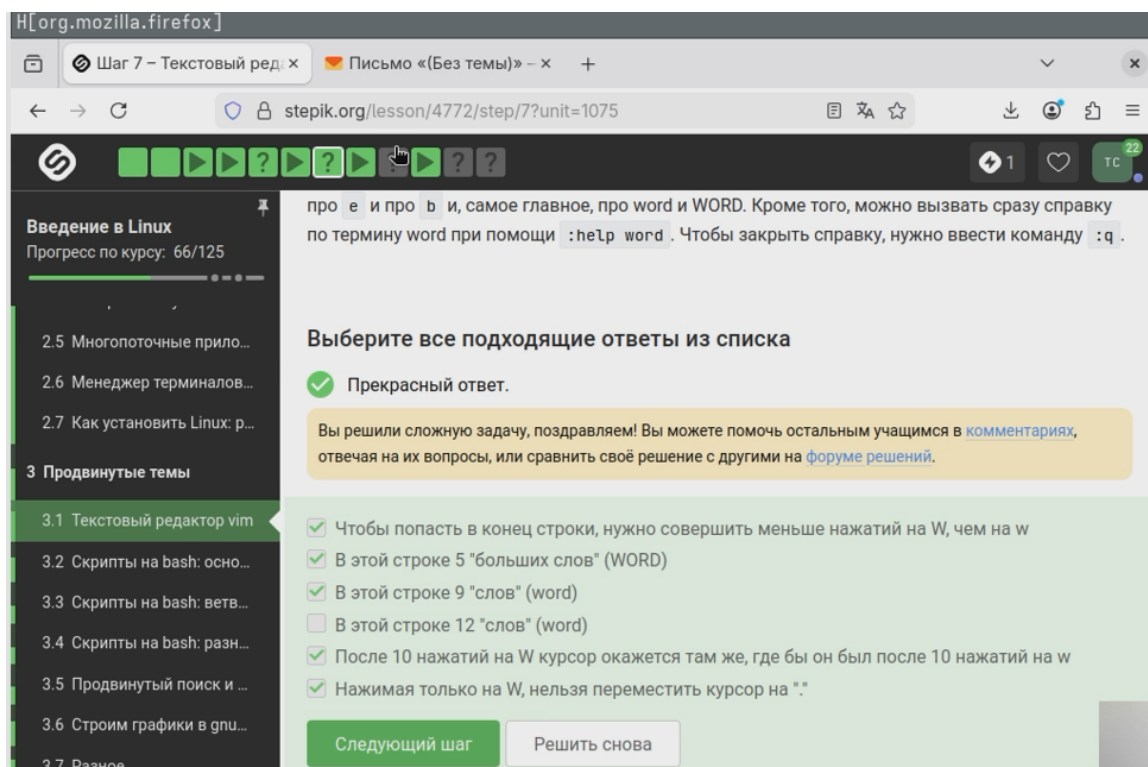


Рисунок 3.1

(ddithree four four four five) — самый прямой: удалить всю строку и вставить новую. (d2w\$bifour four) — удалить первые два слова, перейти в конец строки, вернуться на начало последнего слова и вставить перед ним два four. (d2wwywPp) — удалить первые два слова, перейти на слово four, скопировать его и вставить копию перед оригиналом, получив два four подряд, затем повторить. Остальные варианты приводят к ошибочному результату (рис. 3.2).

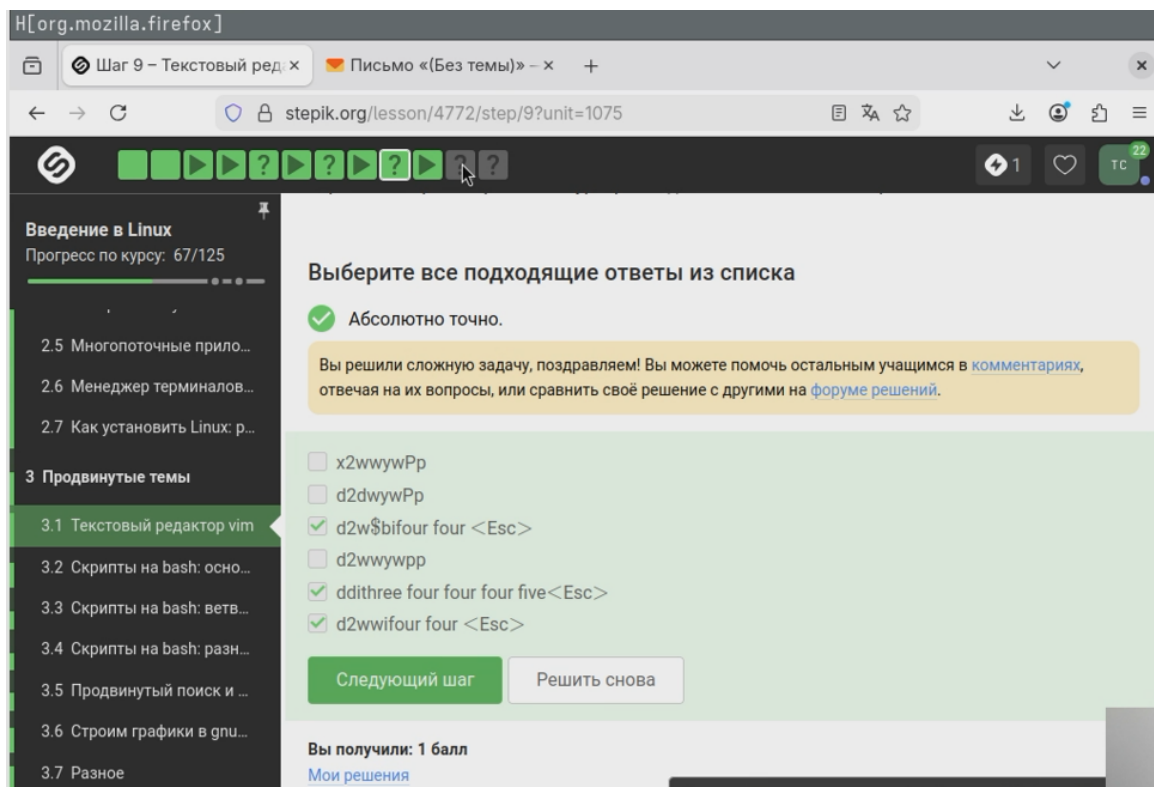


Рисунок 3.2

:%s — замена во всех строках, шаблон Windows, замена Linux. Без флага g заменяется только первое вхождение в строке. Enter не входит в команду (рис. 3.3).

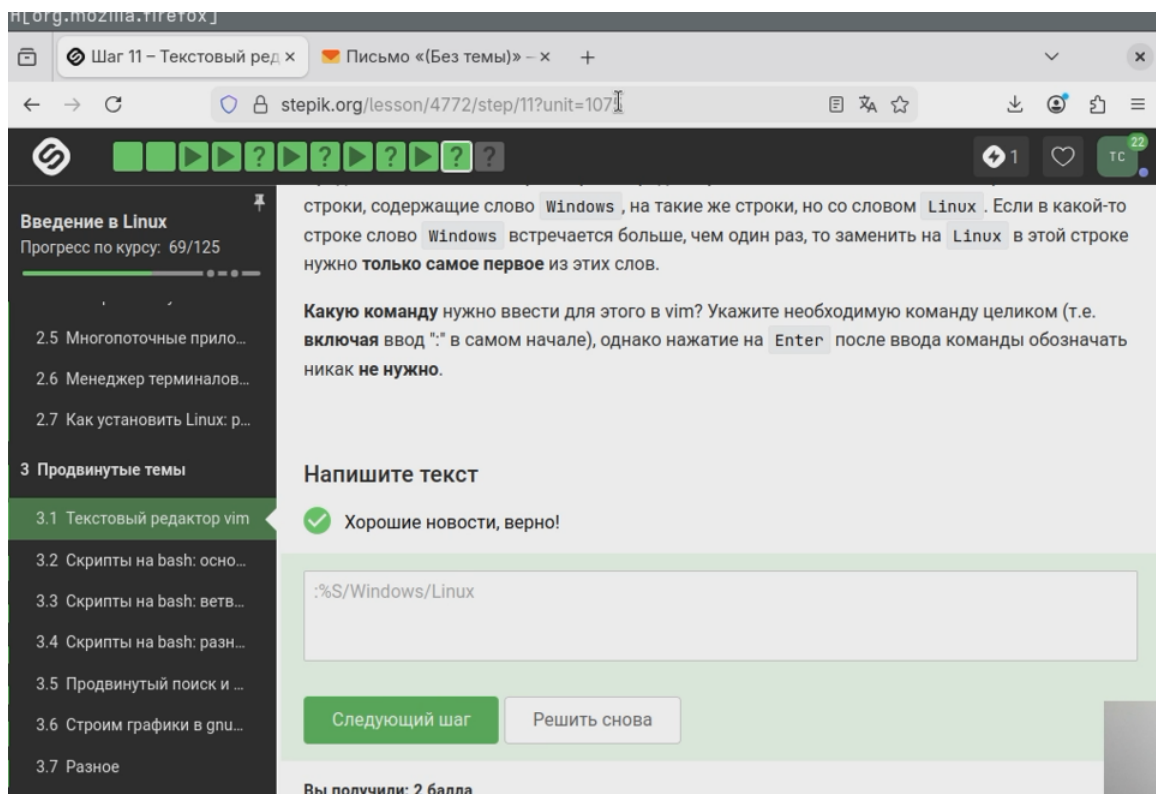


Рисунок 3.3

Режим Visual включается `v` (из нормального, не из любого). Выход — `Esc`, не `:q`. Команда `:visual` не включает его (рис. 3.4).

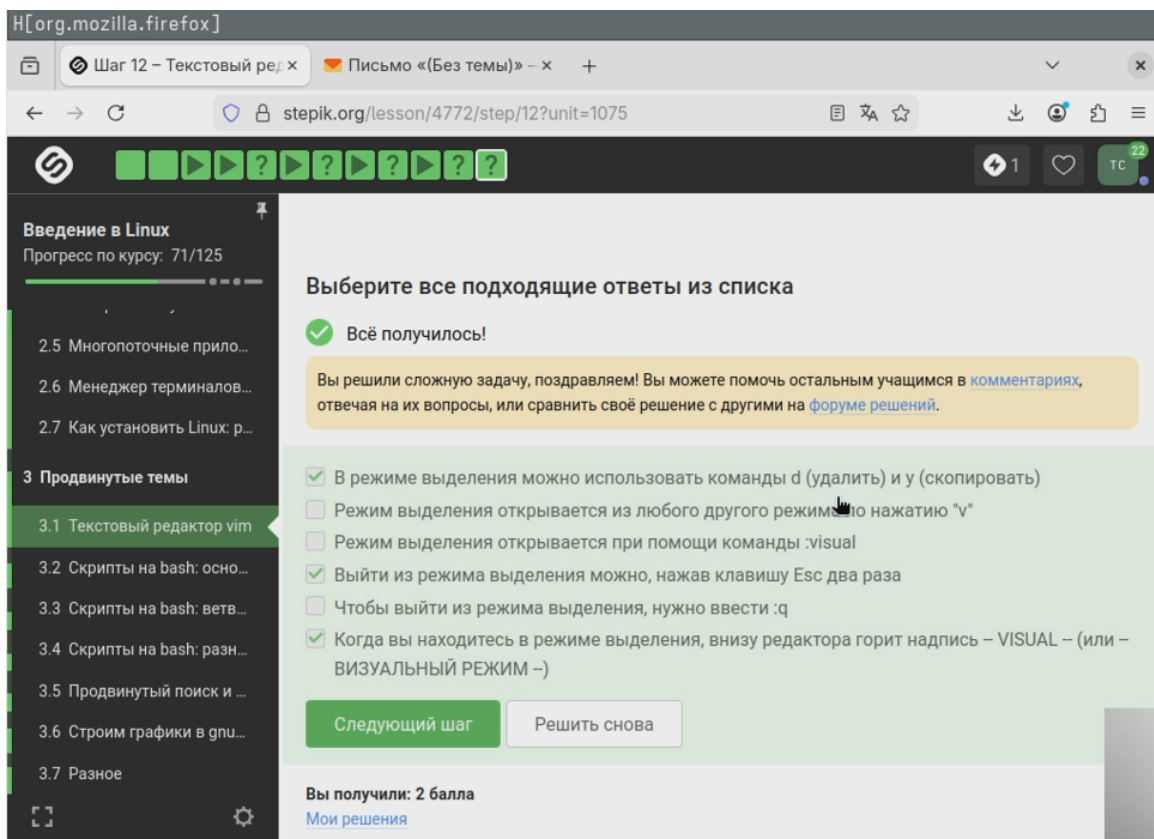


Рисунок 3.4

3.2 Скрипты на bash: основы

История команд привязана к текущей оболочке. Запуск новой оболочки (sh, затем bash) даёт новую историю (рис. 3.5).

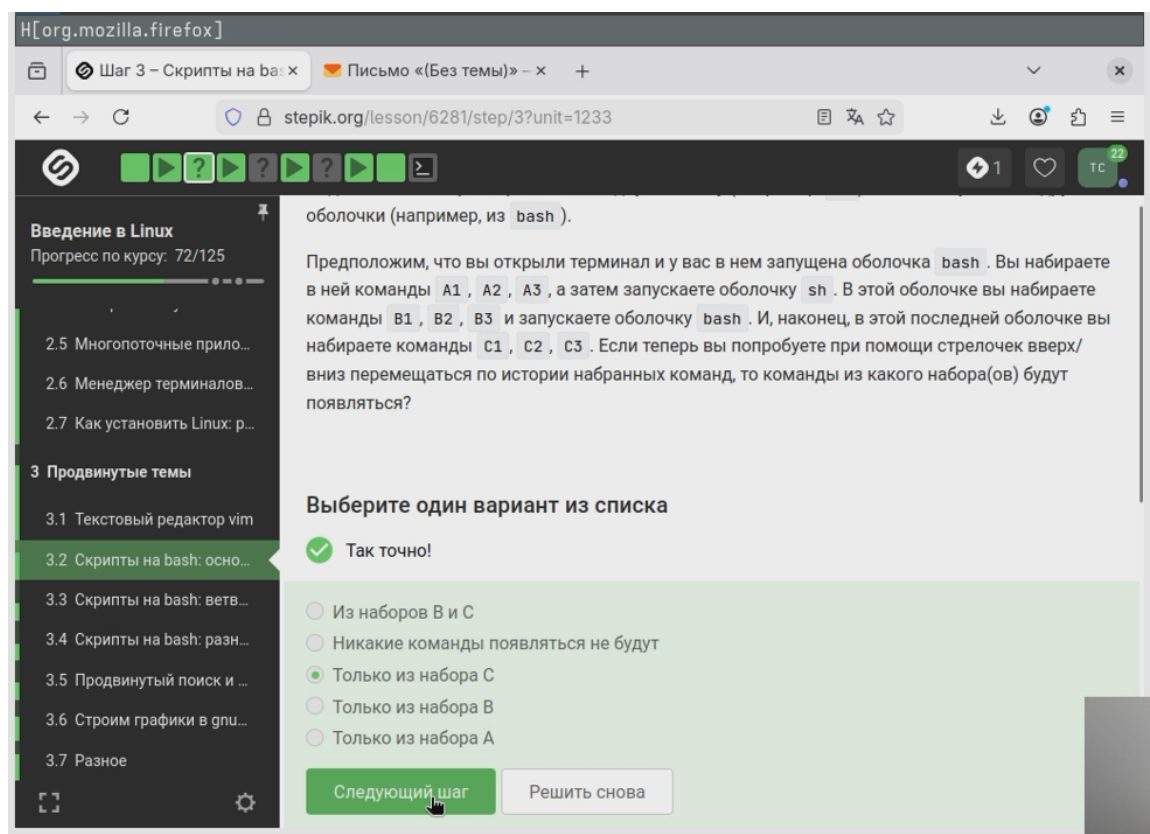


Рисунок 3.5

Команда `cd` меняет директорию только внутри скрипта, но файл создаётся в `/home/bi/` (рис. 3.6).

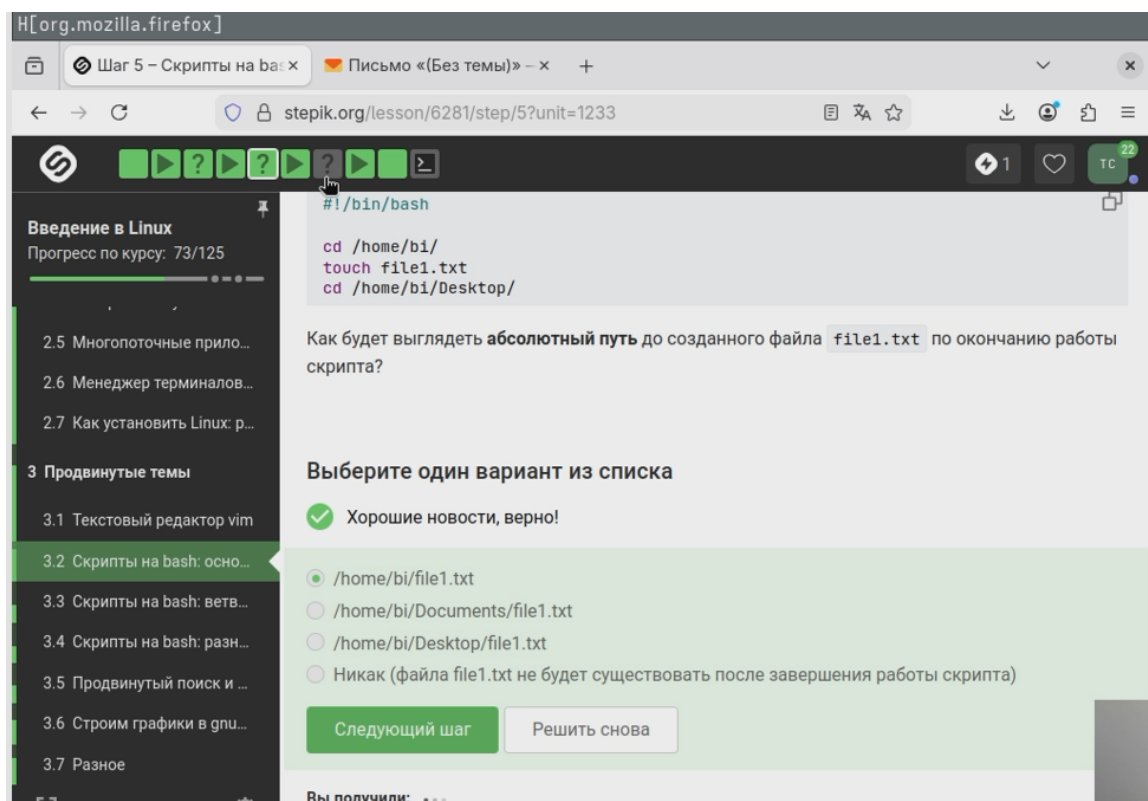


Рисунок 3.6

Имя переменной может содержать буквы, цифры и знак подчёркивания, не может начинаться с цифры. Все перечисленные кроме `vari/able` (слеш недопустим) подходят (рис. 3.7).

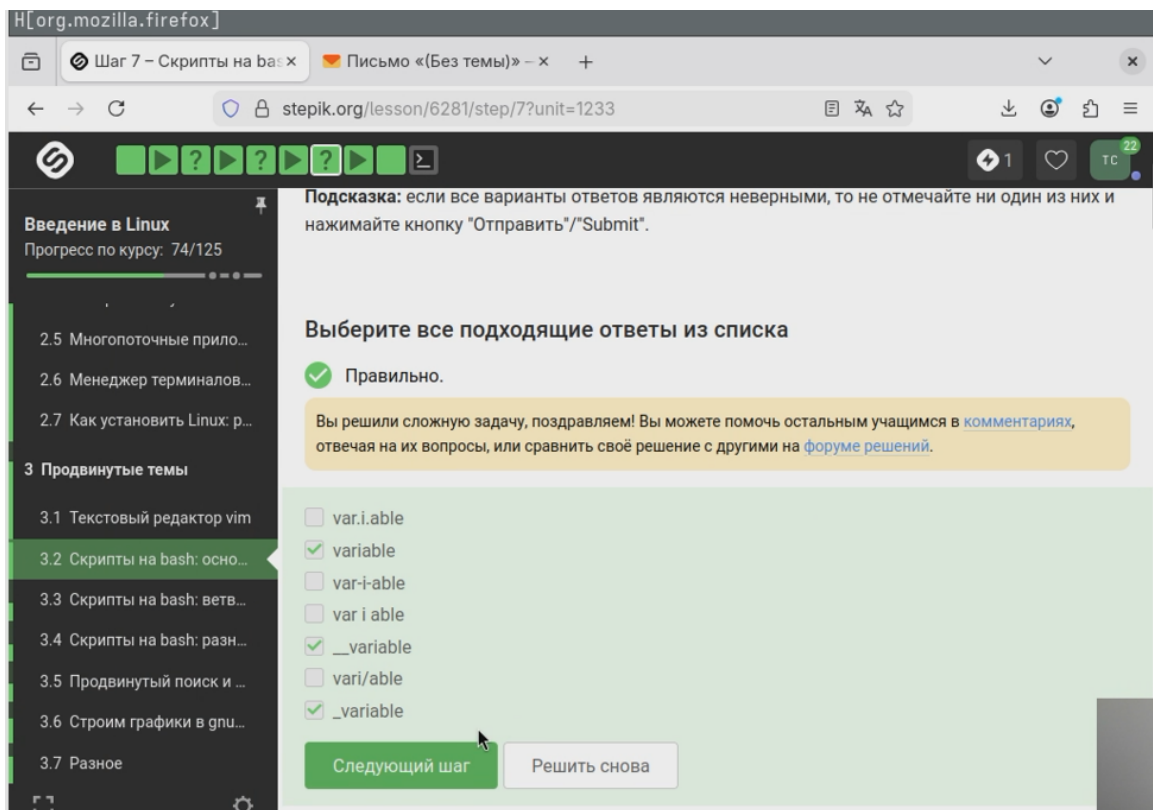


Рисунок 3.7

\$1 и \$2 — аргументы скрипта. Чтобы вывести символ \$ как текст, его нужно экранировать \$. Первый вариант (\$1=\$1) выведет \$1=значение (рис. 3.8).

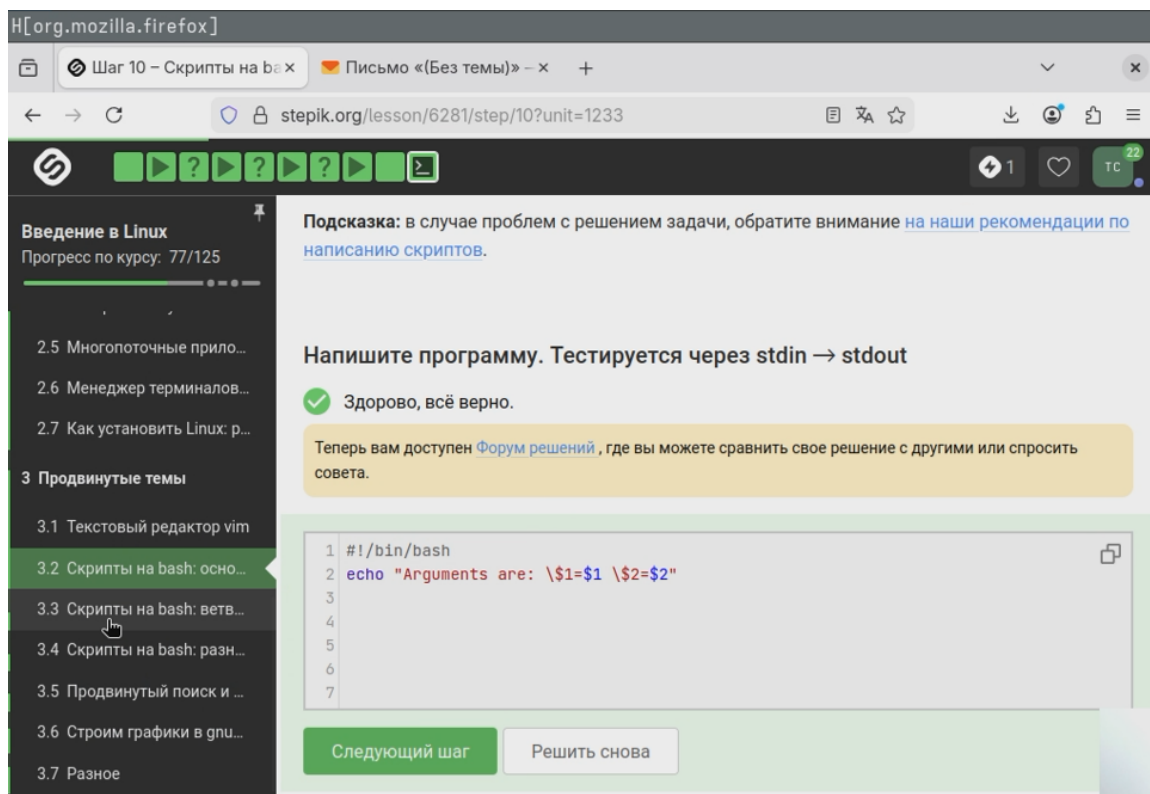


Рисунок 3.8

3.3 Скрипты на bash: ветвления и циклы

Верные: `$# -ge 0` — число аргументов всегда ≥ 0 ; `-z ""` — пустая строка — всегда true ; `! (4 -le 3)` — $4 \leq 3$ — false, отрицание — true ; `5 -ge 5` — всегда true. Остальные зависят от переменных или аргументов (рис. 3.9).

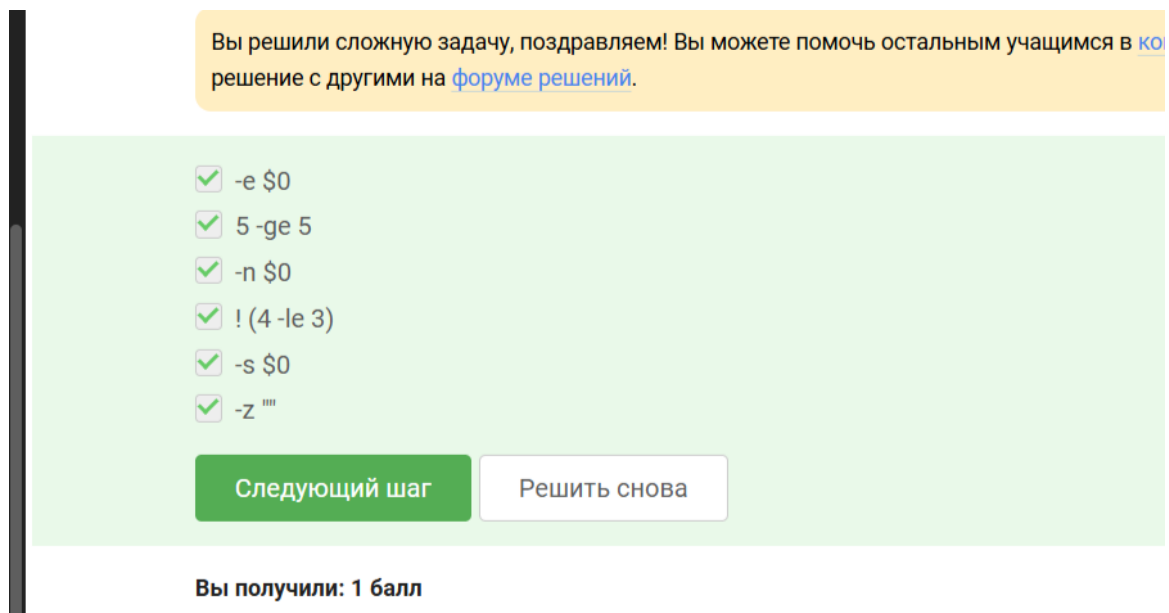


Рисунок 3.9

При `var=3`: первое условие `$var -gt 5` — `false`, второе `$var -lt 3` — `false`, третье `$var -eq 4` — `false` → `else` → `four`. При `var=5`: `-gt 5`? 5 не больше 5 → `false`, `-lt 3` `false`, `-eq 4` `false` → снова `four` (рис. 3.10).

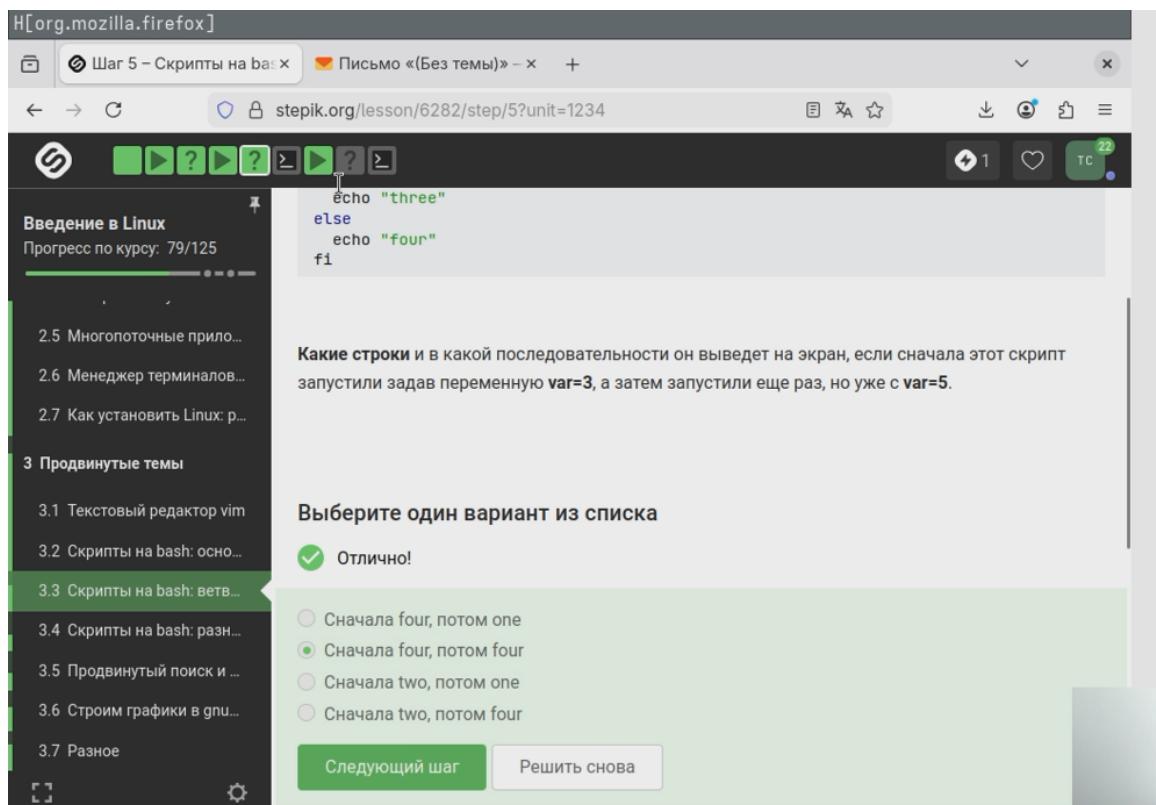


Рисунок 3.10

Конструкция проверяет число студентов и выводит нужное сообщение. Для 2–4 выводится число с «students». Для 0 особая строка. Для 5 и больше — «A lot of students» (рис. 3.11).

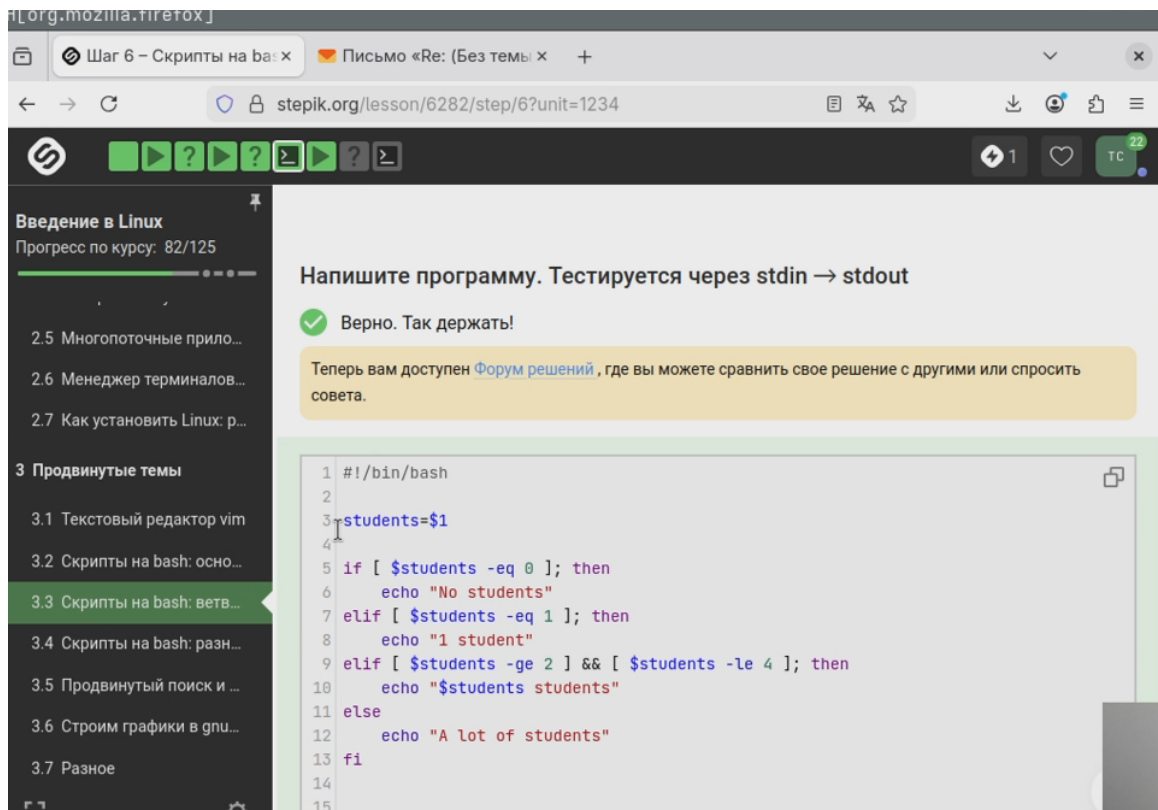


Рисунок 3.11

Цикл `for` перебирает три строки: «a», «b» и «c_d». `continue` пропускает вывод «finish» для строк, которые лексикографически больше «c». «c_d» > «c», поэтому для неё «finish» не выводится. Остальные строки выводят и «start», и «finish». Итог: «start» — 5 раз, «finish» — 4 раза (рис. 3.12).

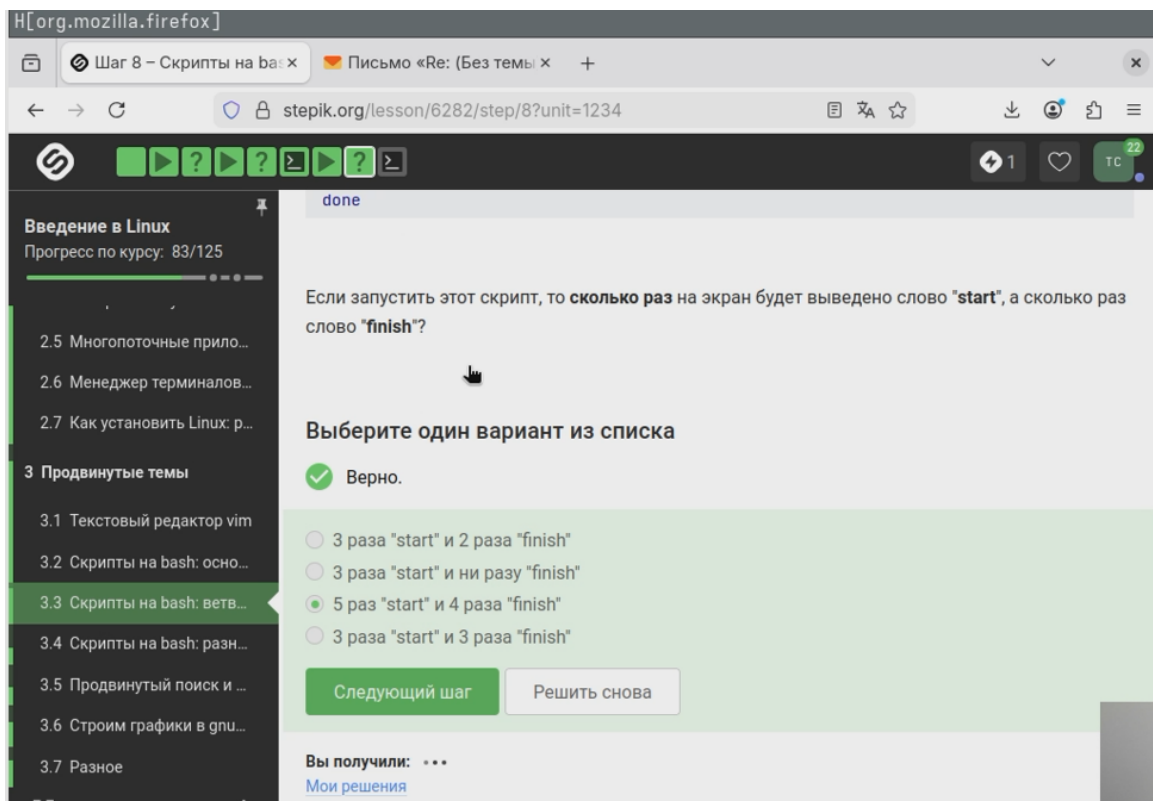
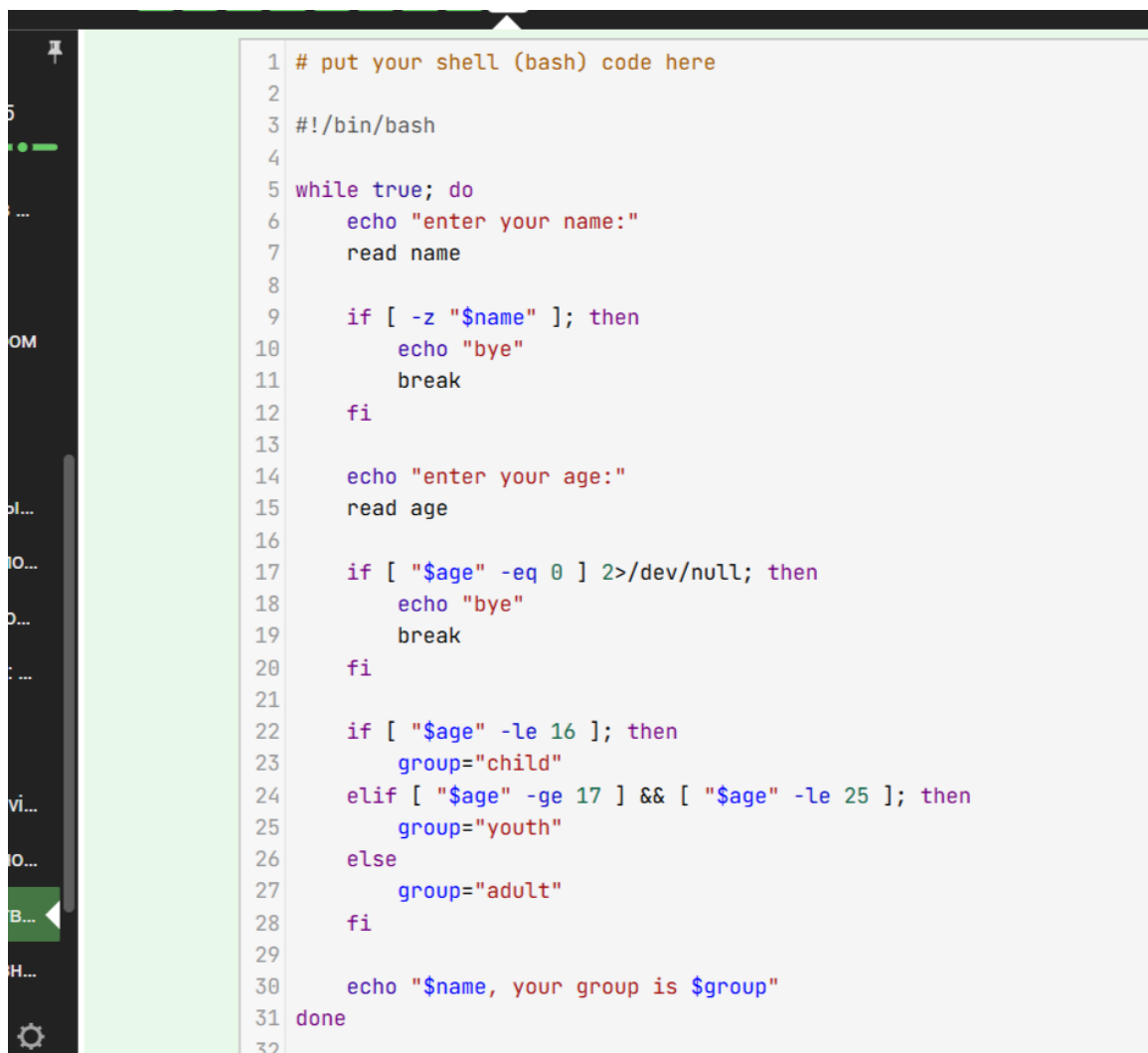


Рисунок 3.12

Бесконечный цикл `while true`. При пустом имени или возрасте 0 — выход с `bye`.
 Иначе определение группы (рис. 3.13).



```
1 # put your shell (bash) code here
2
3 #!/bin/bash
4
5 while true; do
6     echo "enter your name:"
7     read name
8
9     if [ -z "$name" ]; then
10         echo "bye"
11         break
12     fi
13
14     echo "enter your age:"
15     read age
16
17     if [ "$age" -eq 0 ] 2>/dev/null; then
18         echo "bye"
19         break
20     fi
21
22     if [ "$age" -le 16 ]; then
23         group="child"
24     elif [ "$age" -ge 17 ] && [ "$age" -le 25 ]; then
25         group="youth"
26     else
27         group="adult"
28     fi
29
30     echo "$name, your group is $group"
31 done
32
```

Рисунок 3.13

3.4 Скрипты на bash: разное

let выполняет арифметику. Пробелы в кавычках допустимы. $a+=b$ — сокращённая запись (рис. 3.14).

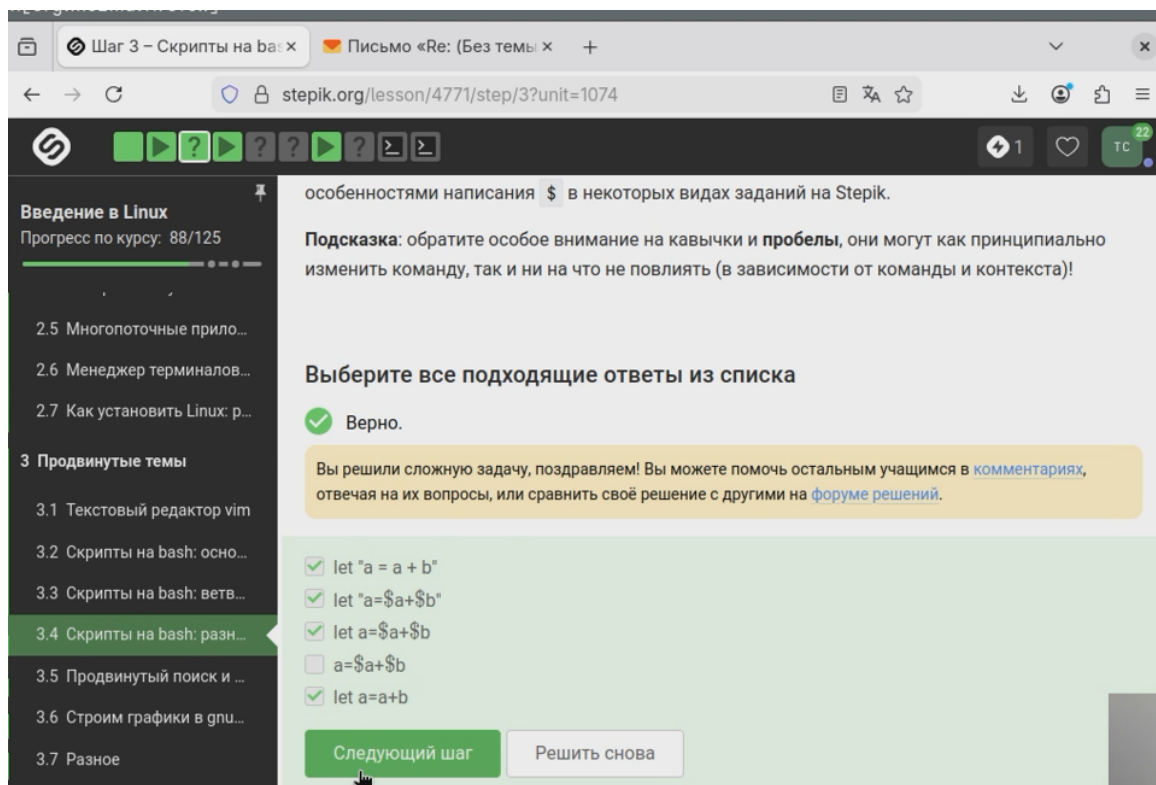


Рисунок 3.14

Команда `echo «pwd»` выводит строку `pwd`, а не результат выполнения `pwd`. Кавычки означают текстовую строку (рис. 3.15).

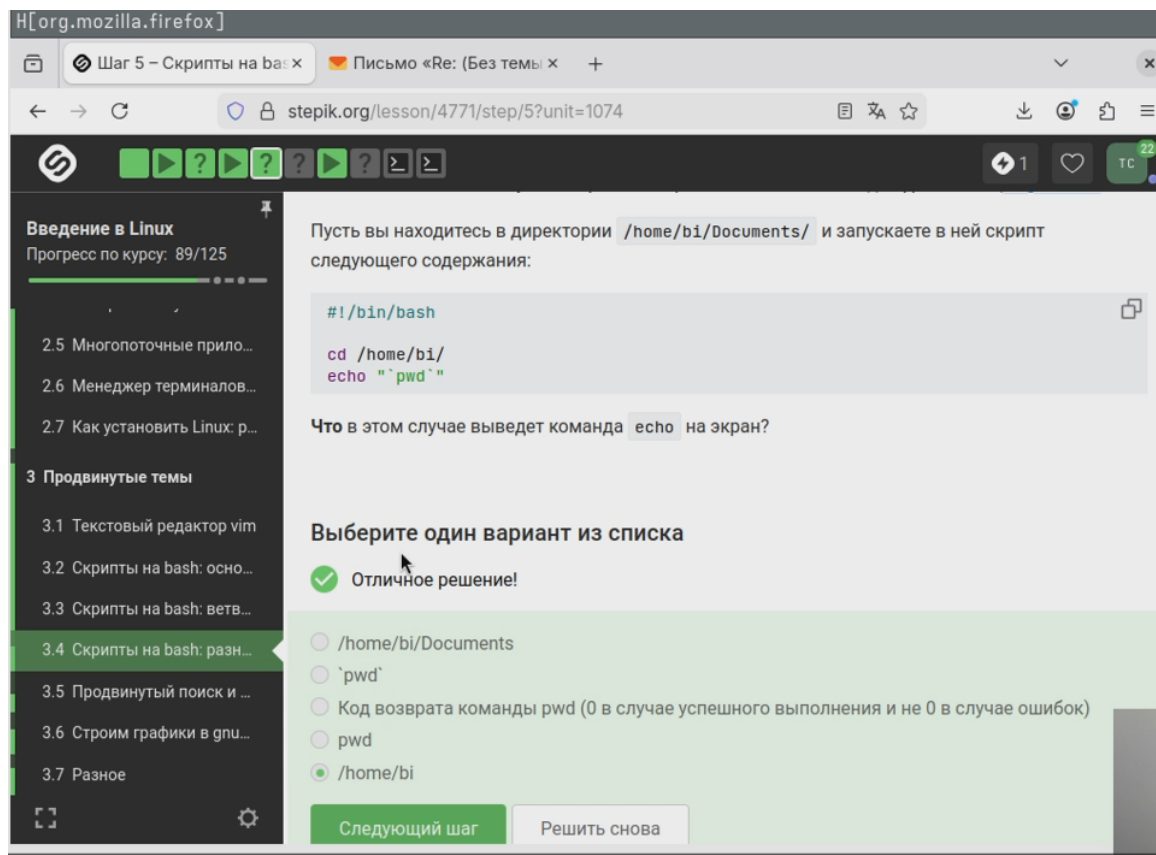


Рисунок 3.15

Чтобы проверить код возврата программы, которая пишет в `stdout`, нужно либо перенаправить её вывод (чтобы он не попал в условие `if`), либо явно проверить переменную `?` после её выполнения. Остальные способы либо некорректны, либо проверяют вывод, а не код возврата (рис. 3.16).

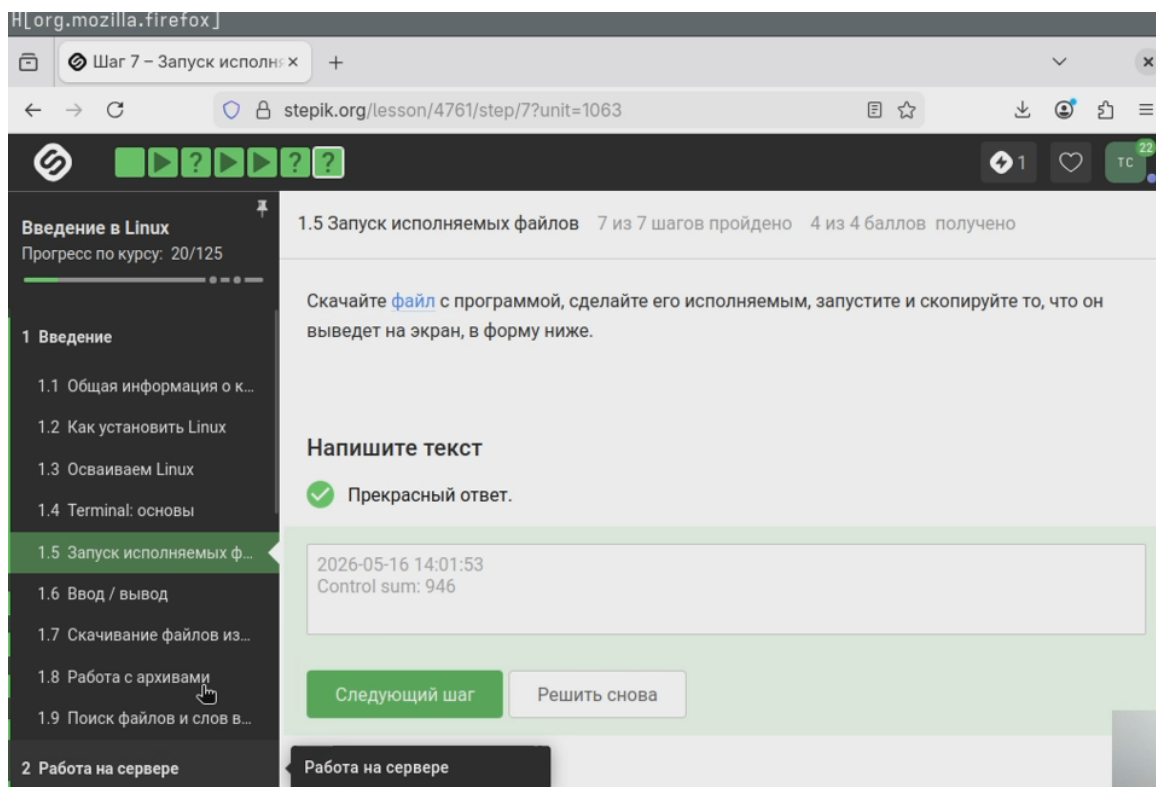


Рисунок 3.16

В функции counter переменная `s1` объявлена как `local`, поэтому её значение не сохраняется между вызовами — после десяти вызовов `s1` остаётся пустой. Нелокальная переменная `s2` накапливает сумму: $2 \cdot (1+2+\dots+10)=110$. Команда `echo` выводит «counters are and 110» (между `are` и `and` — два пробела) (рис. 3.17).

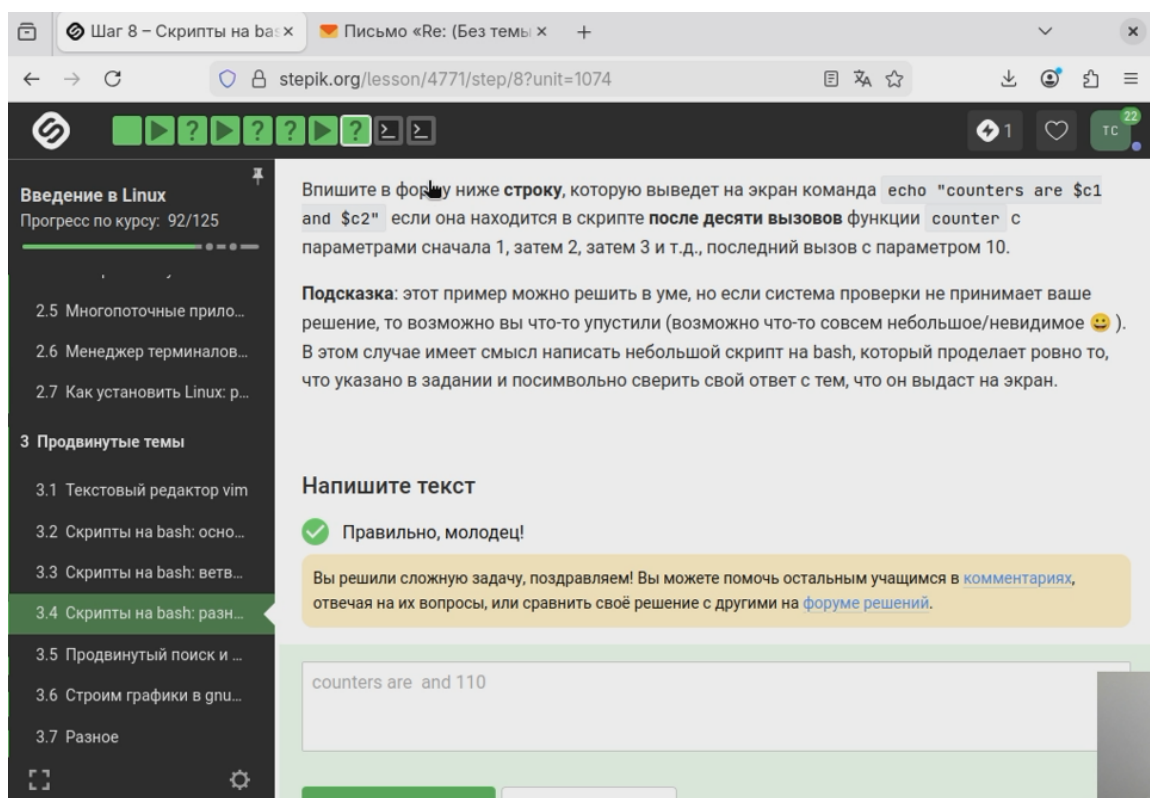
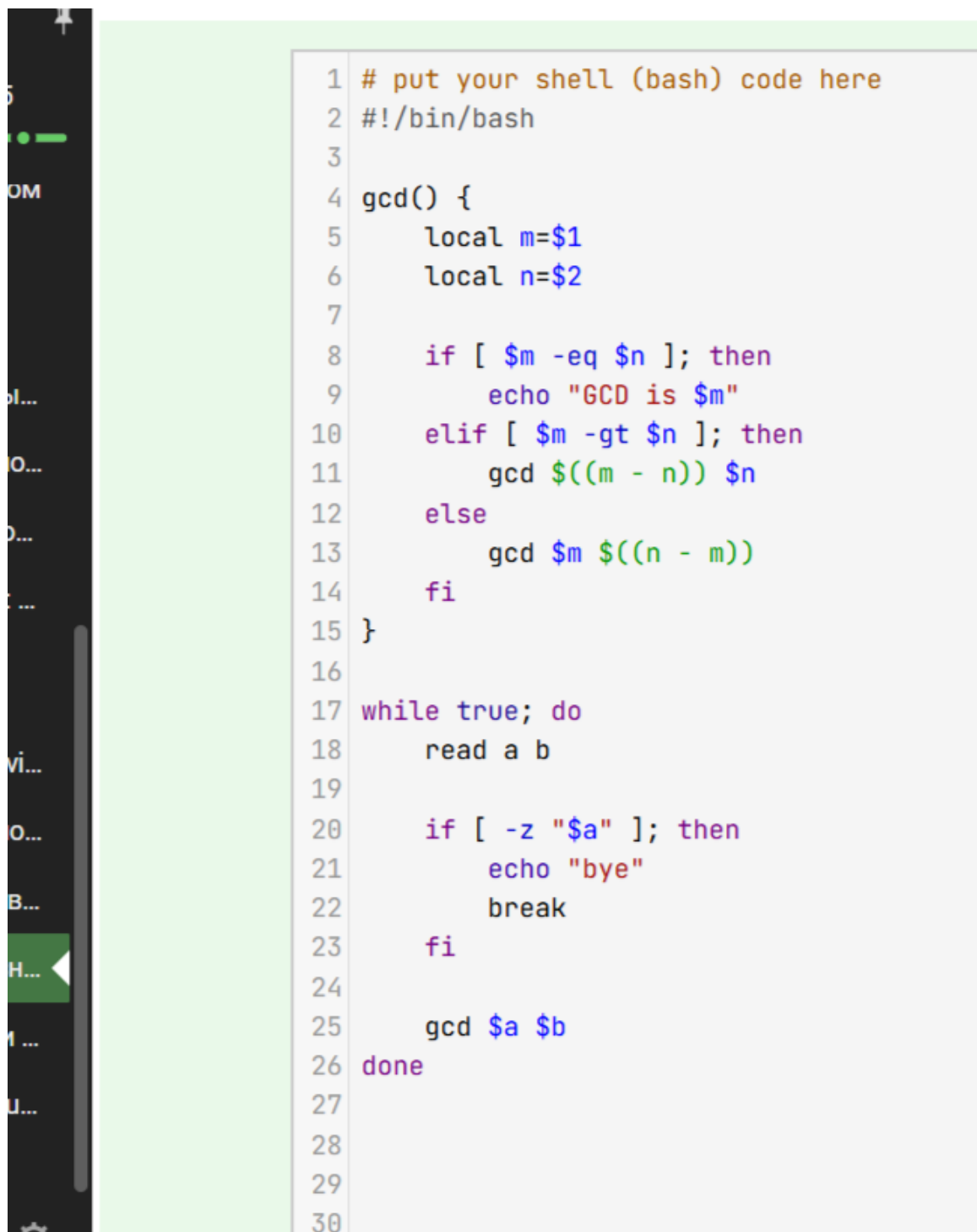


Рисунок 3.17

Бесконечный цикл чтения двух чисел. Пустая строка — выход. Функция gcd реализует алгоритм Евклида (рис. 3.18).



```
1 # put your shell (bash) code here
2 #!/bin/bash
3
4 gcd() {
5     local m=$1
6     local n=$2
7
8     if [ $m -eq $n ]; then
9         echo "GCD is $m"
10    elif [ $m -gt $n ]; then
11        gcd $((m - n)) $n
12    else
13        gcd $m $((n - m))
14    fi
15 }
16
17 while true; do
18     read a b
19
20     if [ -z "$a" ]; then
21         echo "bye"
22         break
23     fi
24
25     gcd $a $b
26 done
27
28
29
30
```

Рисунок 3.18

`$((exp))` вычисляет арифметическое выражение. Если выражение неверно —

ошибка и выход. exit — выход с bye (рис. 3.19).

```

1 # put your shell (bash) code here
2 #!/bin/bash
3
4 while true; do
5     read a op b
6
7     if [ -z "$a" ]; then
8         continue
9     fi
10
11     if [ "$a" = "exit" ]; then
12         echo "bye"
13         break
14     fi
15
16     if [ -z "$op" ] || [ -z "$b" ]; then
17         echo "error"
18         break
19     fi
20
21     case $op in
22         "+")
23             echo $((a + b))
24             ;;
25         "-")
26             echo $((a - b))
27             ;;
28         "*")
29             echo $((a * b))
30             ;;
31         "/" )
32             if [ $b -eq 0 ]; then
33                 echo "error"
34                 break
35             fi
36             echo $((a / b))
37             ;;
38         "%")
39             if [ $b -eq 0 ]; then
40                 echo "error"
41                 break
42             fi
43             echo $((a % b))
44             ;;
45         "**")
46             echo $((a ** b))
47             ;;
48         *)
49             echo "error"
50             break
51             ;;
52     esac
53 done
54

```

Рисунок 3.19

3.5 Продвинутый поиск и редактирование

-iname ищет без учёта регистра, -name — с учётом (рис. 3.20).

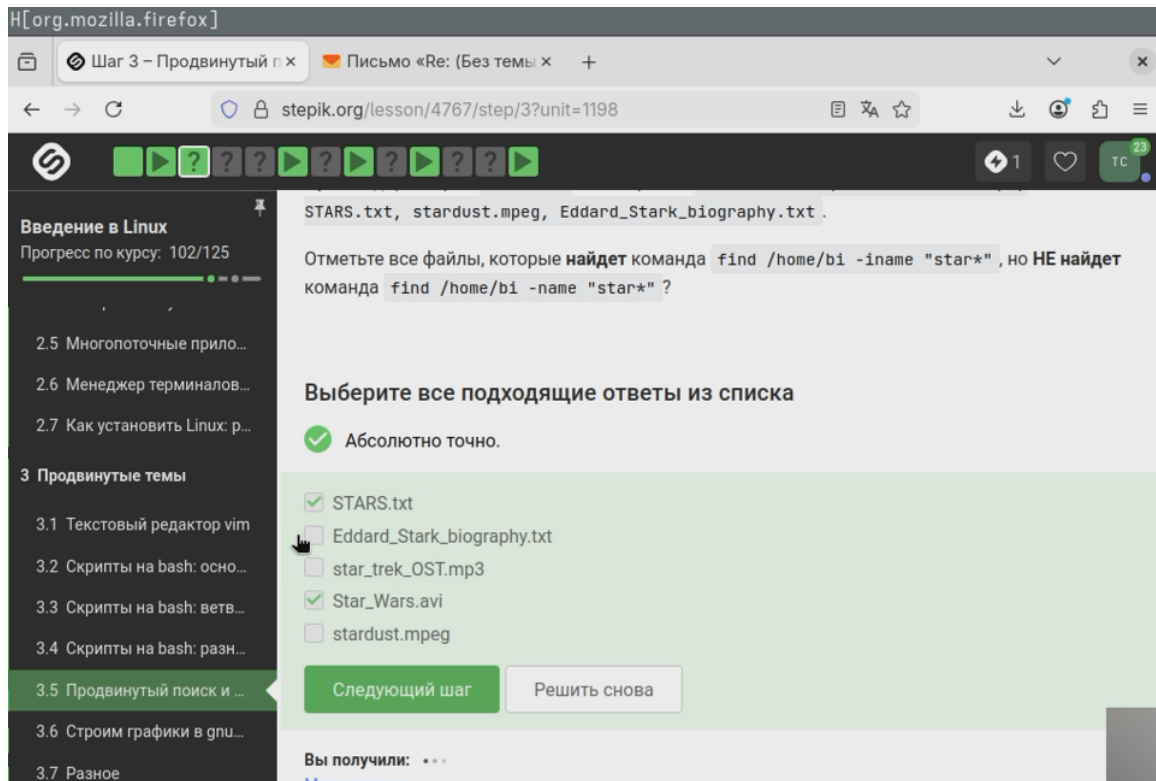


Рисунок 3.20

-path ищет по полному пути, -name только по имени. В некоторых случаях (файл в текущей директории) результат совпадёт, но не всегда (рис. 3.21).

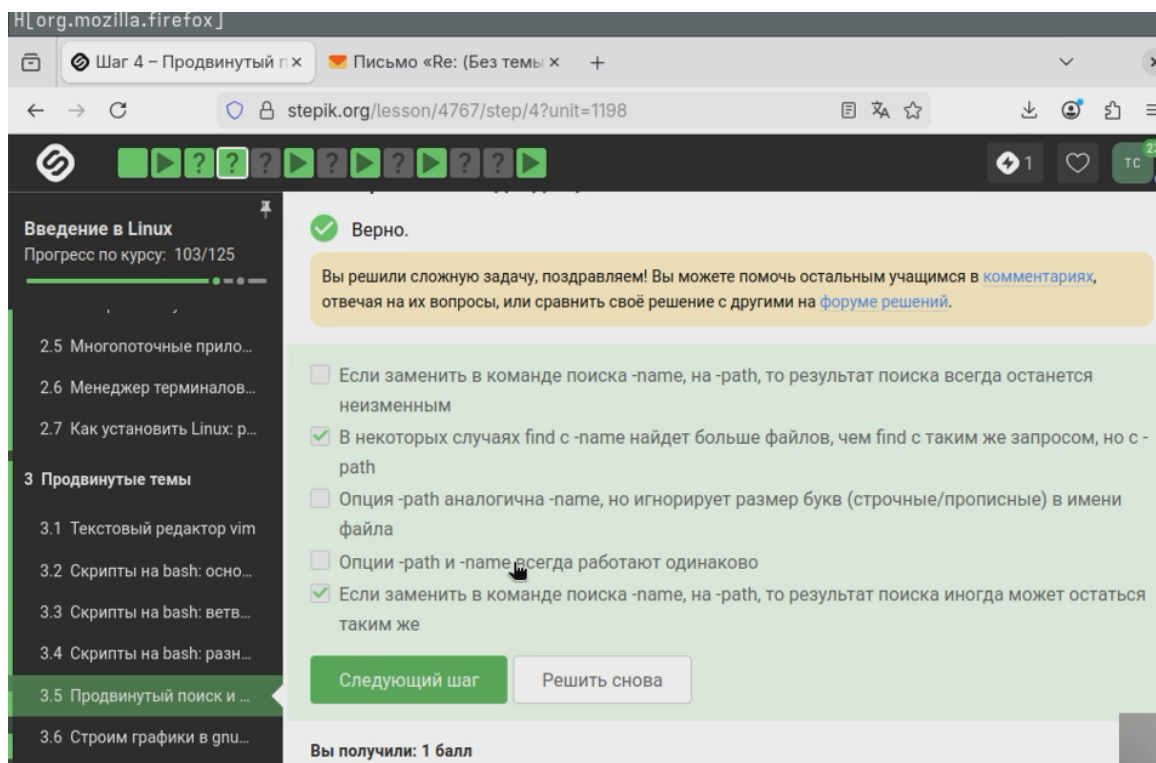


Рисунок 3.21

Команда `find` с параметрами `-mindepth 2 -maxdepth 3` ищет файлы на глубине 2 и 3 относительно `/home/bi/`. `file1` находится на глубине 2 (`/home/bi/dir1/file1`), `file2` — на глубине 3 (`/home/bi/dir1/dir2/file2`), `file3` — на глубине 4 (`/home/bi/dir1/dir2/dir3/file3`). Будут найдены `file1` и `file2` — все, кроме `file3` (рис. 3.22).

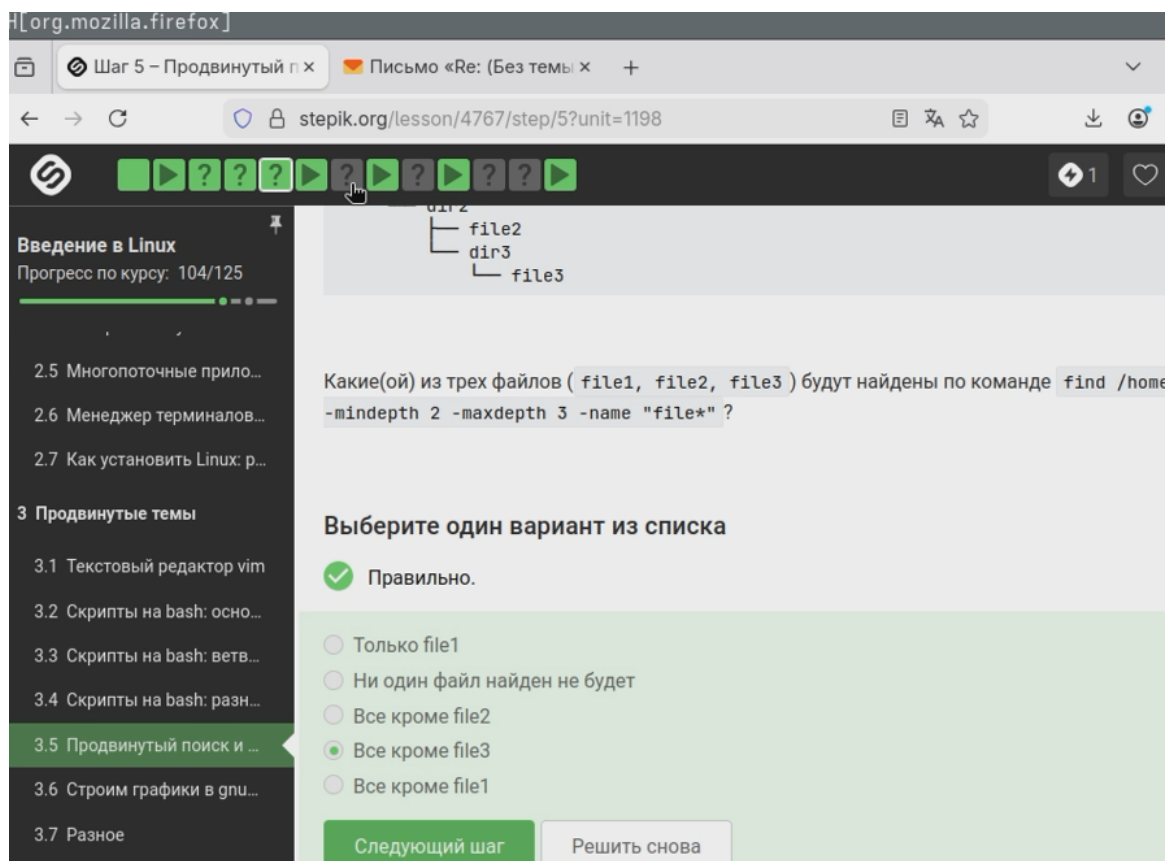


Рисунок 3.22

Во всех вариантах `grep` выводит одинаковое количество строк, так как каждая строка файла содержит слово «word». Опции `-A`, `-B`, `-C` добавляют контекст, но поскольку искомая строка есть в каждой строке, добавленные строки контекста совпадают с уже выводимыми. В итоге все команды создадут файл `results.txt` одинакового размера (рис. 3.23).

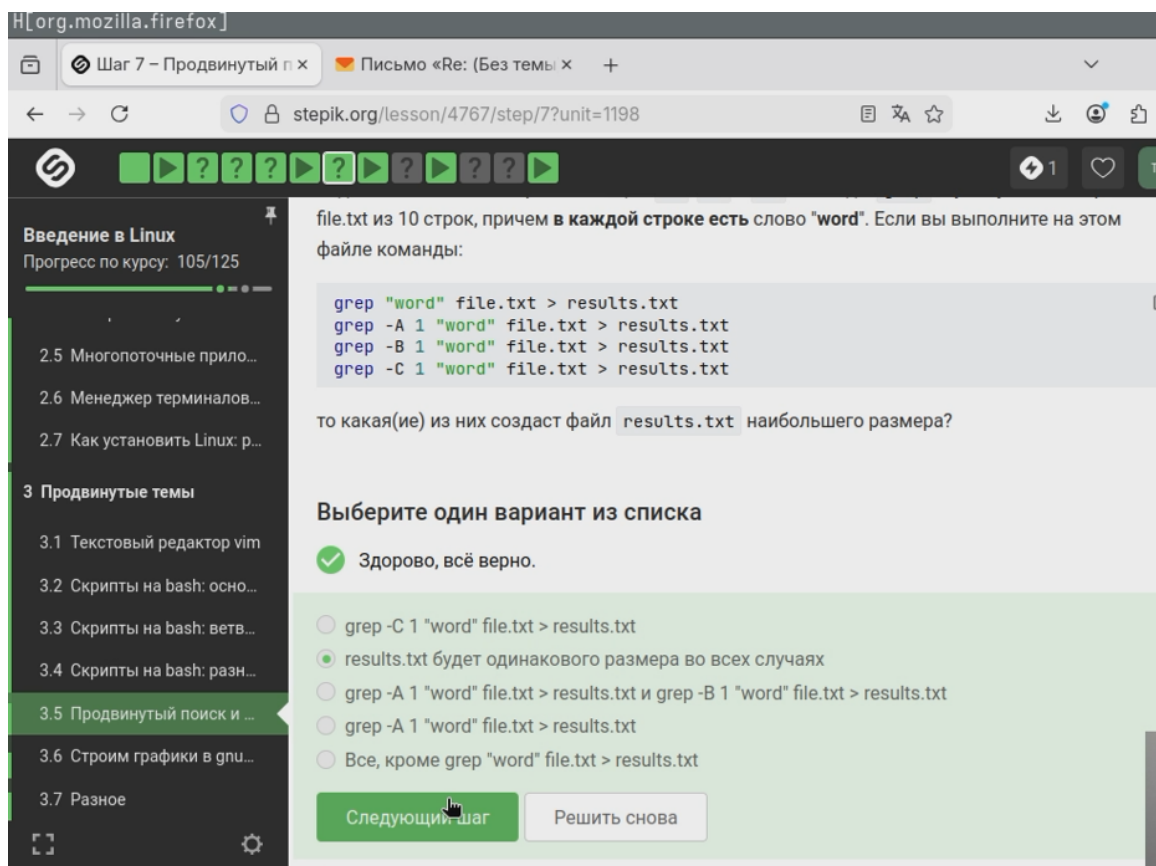


Рисунок 3.23

Регулярное выражение ищет строки, которые оканчиваются на `buntu`. Перед `buntu` может быть буква `u` или цифра `0`, а перед ними — необязательный символ из указанного набора. Строка `Ubuntu!` не подходит из-за восклицательного знака в конце, а `Kbuntu` — потому что после `K` идёт `b`, а не `u` или `0` (рис. 3.24).

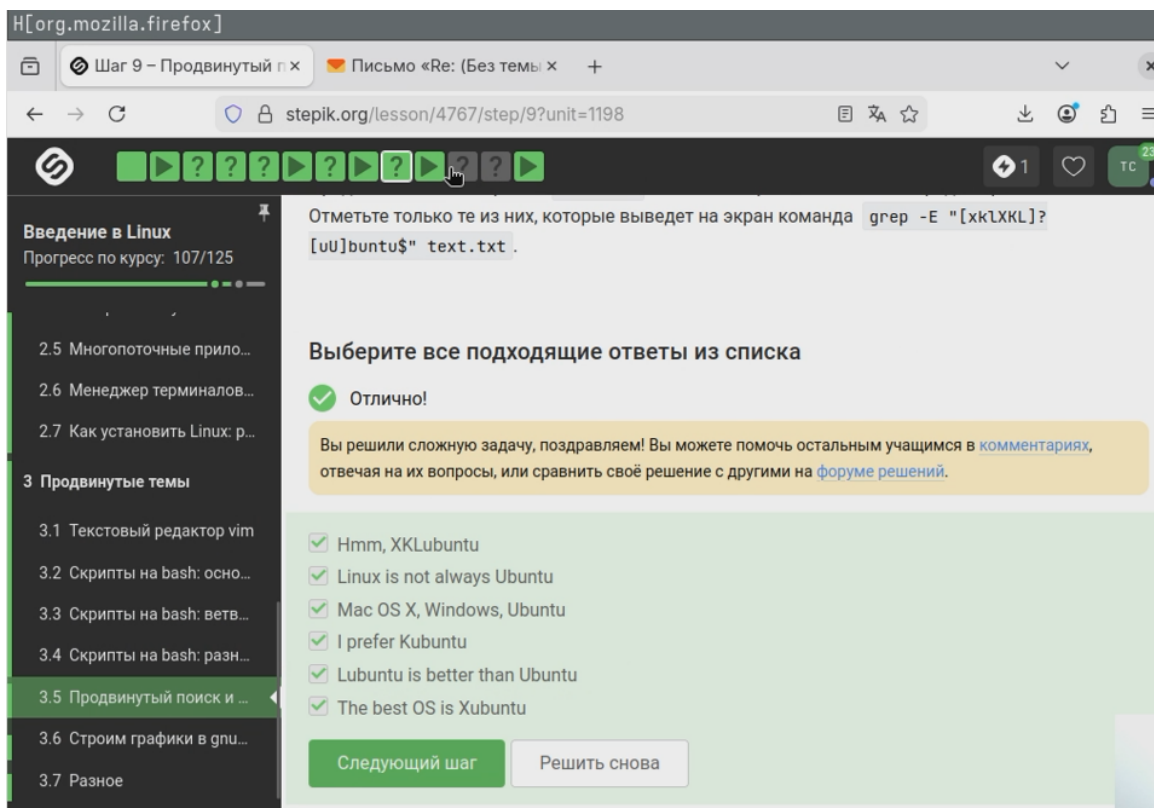


Рисунок 3.24

Без опции `-n` `sed` печатает каждую строку по умолчанию, а команда `p` печатает её ещё раз. Строки, соответствующие шаблону `[a-z]*` (хотя бы одна строчная буква), выводятся дважды. Если строка не соответствует шаблону — печатается один раз (рис. 3.25).

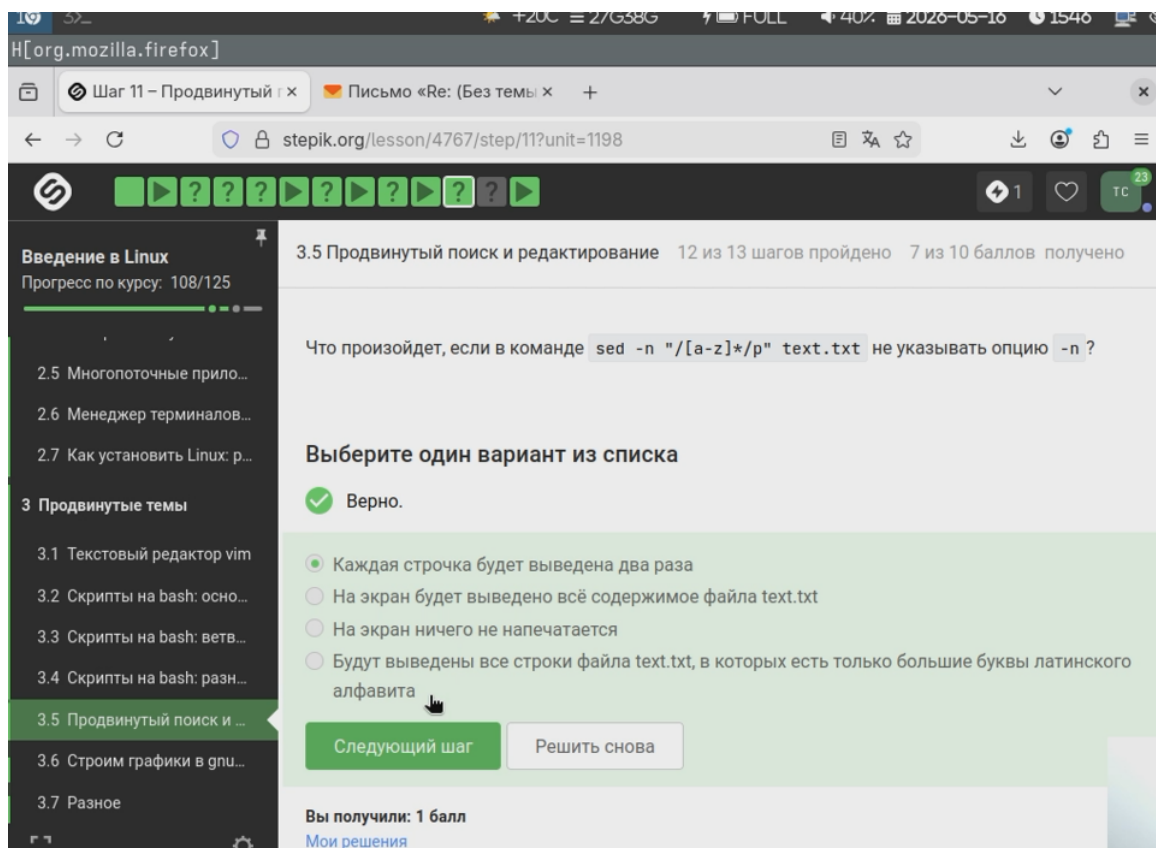


Рисунок 3.25

[A-Z]{2,} — две и более заглавные буквы. g — глобально. Пробелы сохраняются, т.к. мы заменяем только саму аббревиатуру (рис. 3.26).

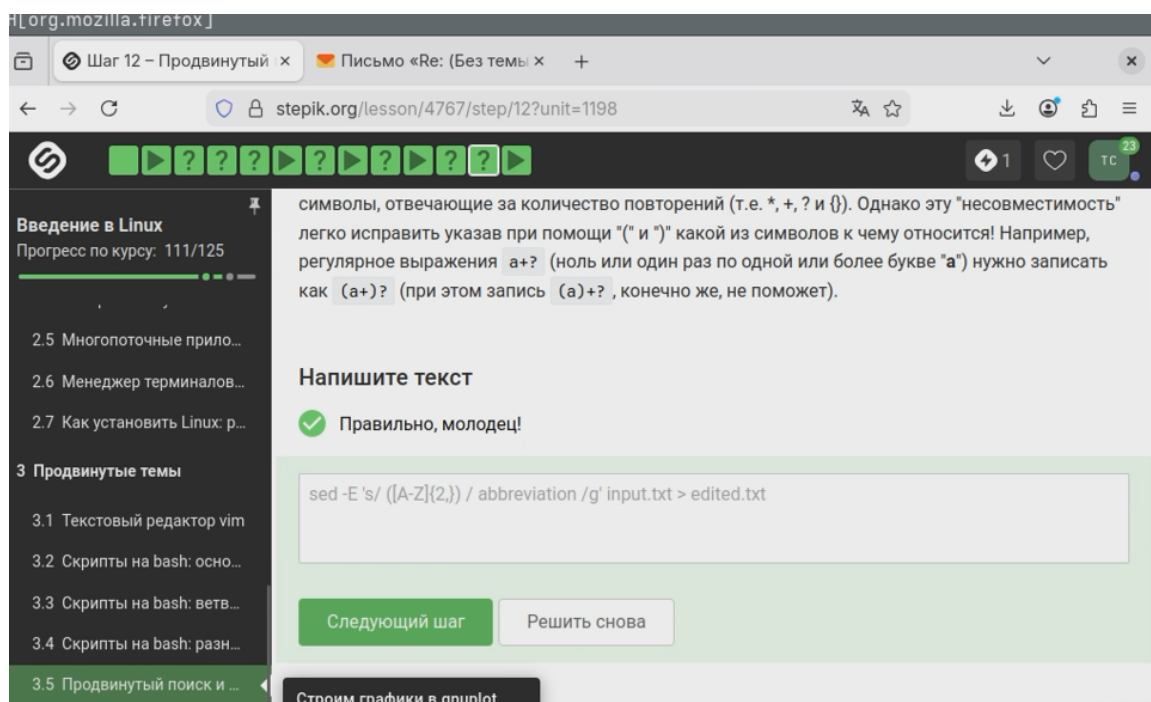


Рисунок 3.26

3.6 Строим графики в gnuplot

Опция `-p` (`-persist`) в `gnuplot` предотвращает закрытие окон с графиками после завершения программы. Это удобно, когда нужно просмотреть график после выхода из `gnuplot` (рис. 3.27).

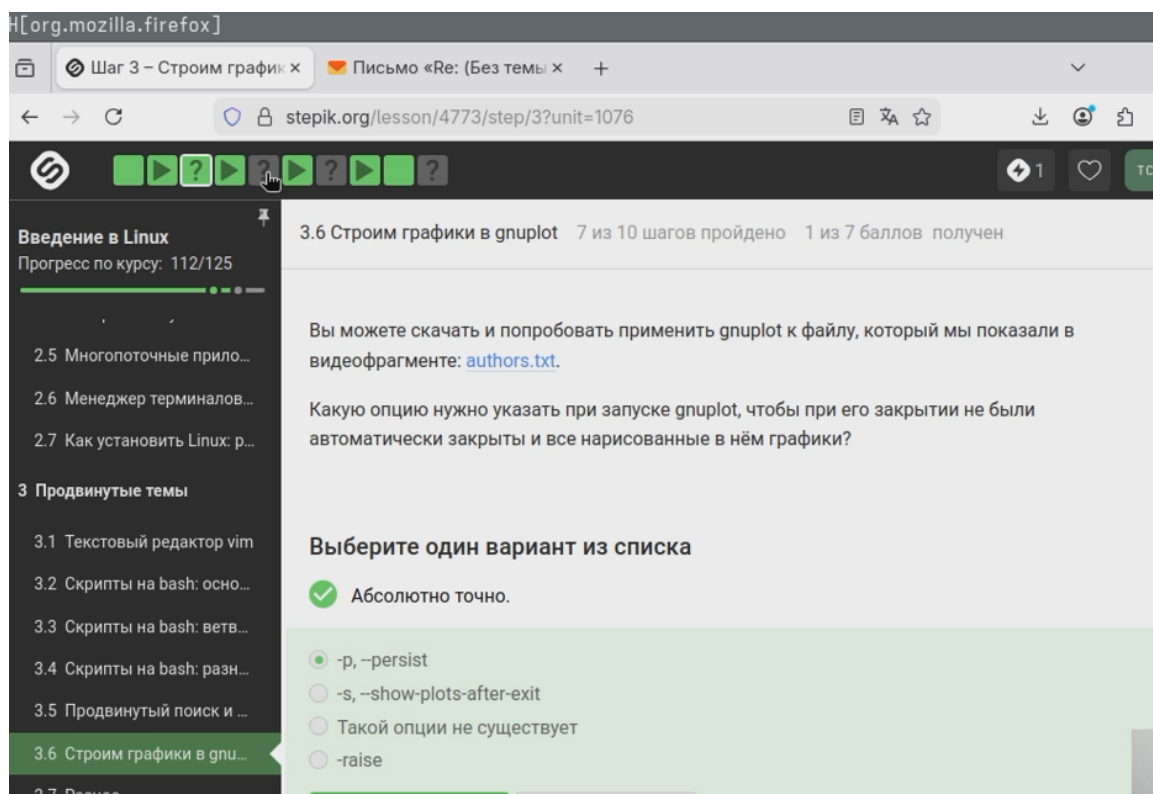


Рисунок 3.27

Опция `columnhead` интерпретирует первую строку файла как заголовок. Если в первой строке находятся данные, то: Значение из первого столбца (или второго, в зависимости от `using`) становится названием ряда ; Сама эта строка в построении графика не участвует, поэтому точек становится на одну меньше, чем строк в файле (рис. 3.28).

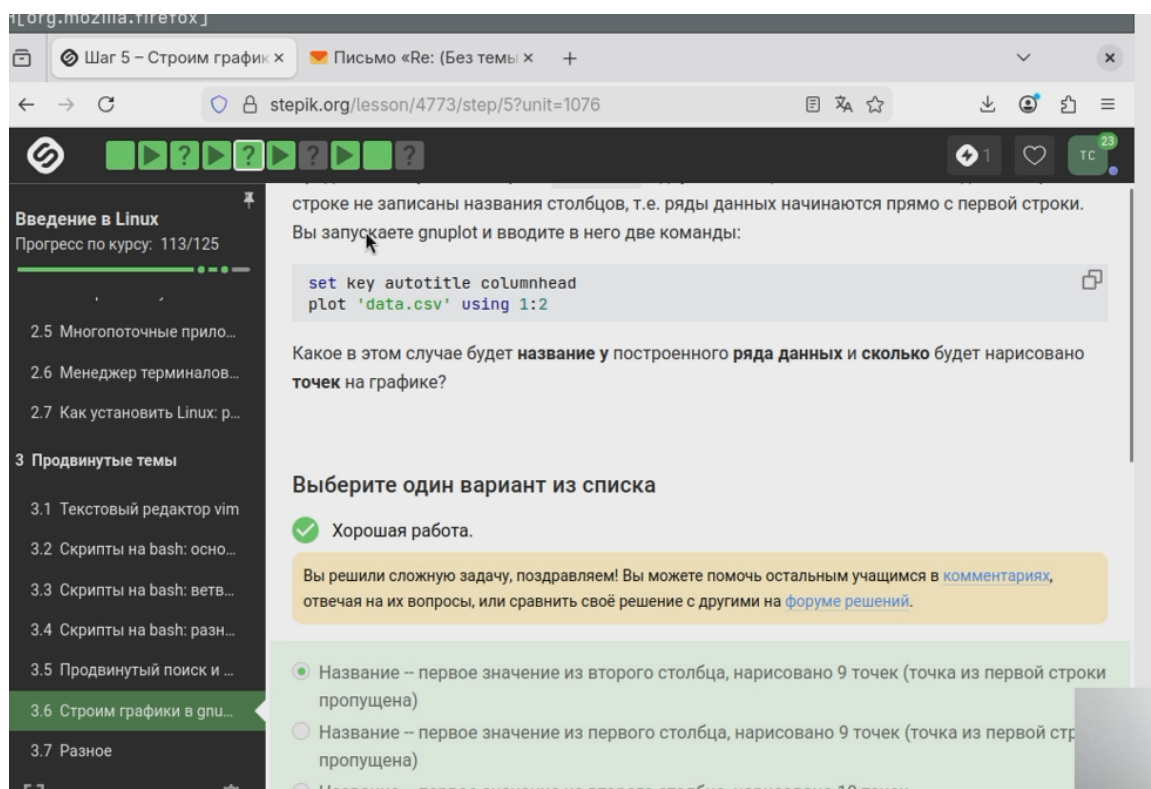


Рисунок 3.28

Команда `set xtics` создаёт пользовательские деления на оси X. Конкатенация строк выполняется через точку. Например, «point 1, value» . x1 при $x1=0$ даёт «point 1, value 0», после чего указывается координата $x1$. В итоге получаются три метки в нужных местах (рис. 3.29).

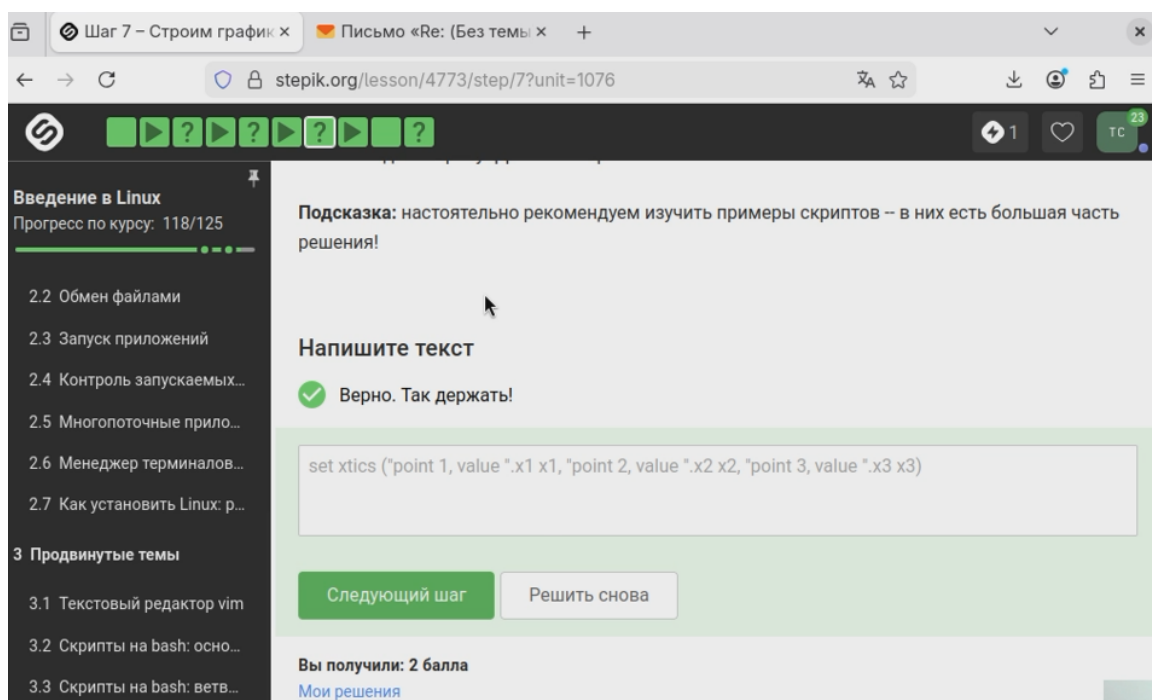


Рисунок 3.29

Для отражения графика относительно горизонтальной плоскости функция изменена с $x^2 + y^2$ на $-x^2 - y^2$. Обратное вращение достигается изменением приращения угла: $zrot = (zrot + 350) \% 360$ (350 эквивалентно -10). Ускорение в два раза — уменьшением паузы с 0.2 до 0.1 секунды (рис. 3.30).

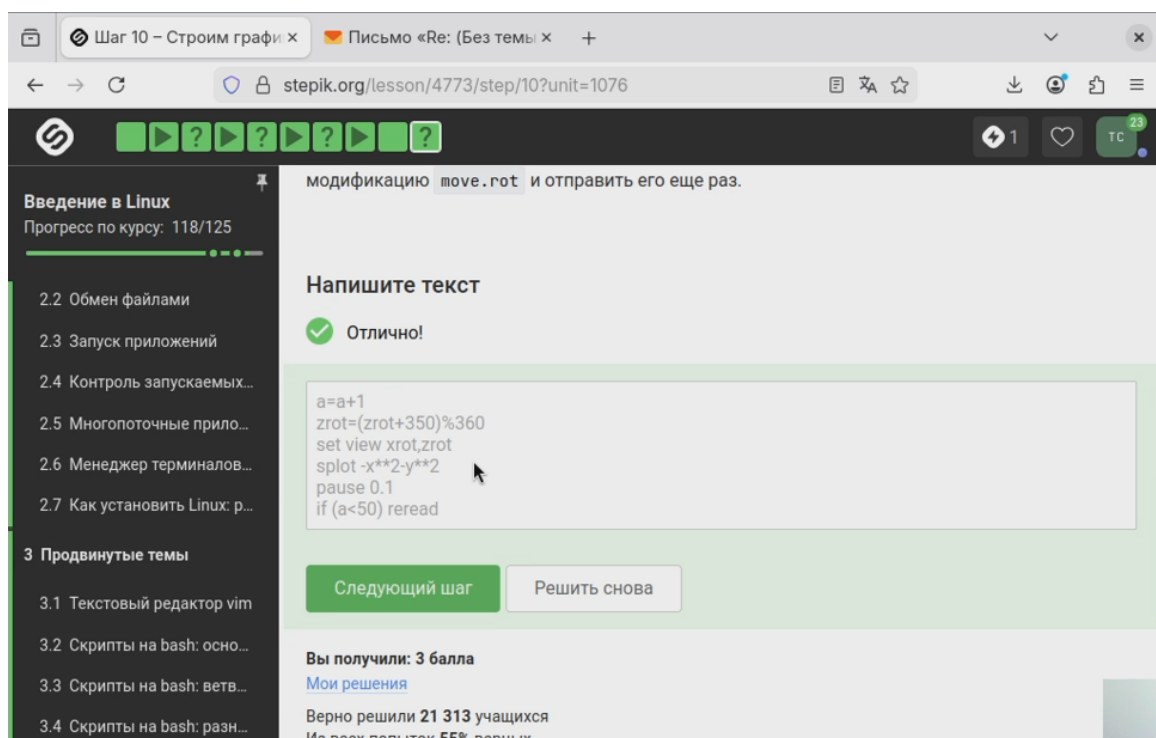


Рисунок 3.30

3.7 Разное

764 в восьмеричной: 7=rwx, 6=rw-, 4=r-. Итог: rwxr-xr-. Другие варианты либо неверные, либо последовательности команд дают другой результат (рис. 3.31).

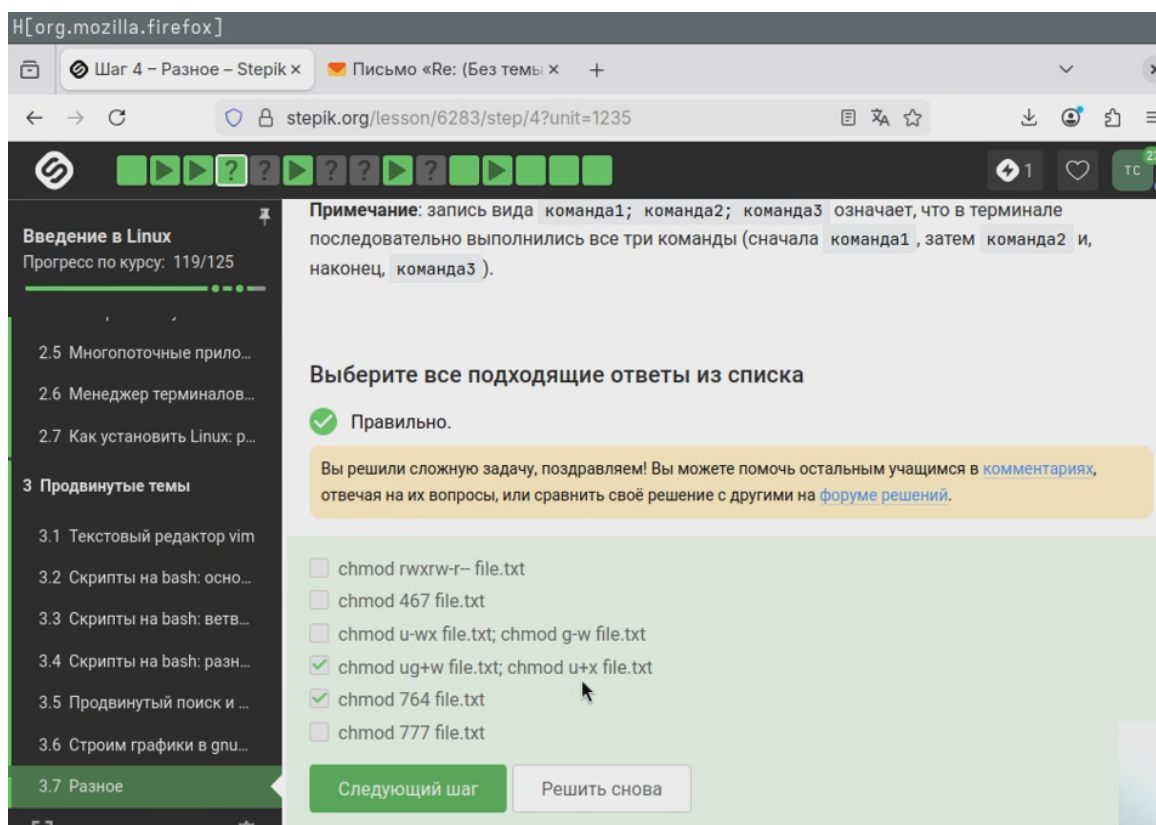


Рисунок 3.31

Чтобы пользователь мог создавать файлы в директории, которая ему не принадлежит, нужно либо сделать его владельцем (`chown`), либо дать право на запись для его категории пользователей (`g+w`, если он в группе, или `o+w`, если он в категории «остальные»). В данном случае `sudo chmod o+w dir` даёт право на запись всем, включая нужного пользователя, поэтому этот вариант также верен (рис. 3.32).

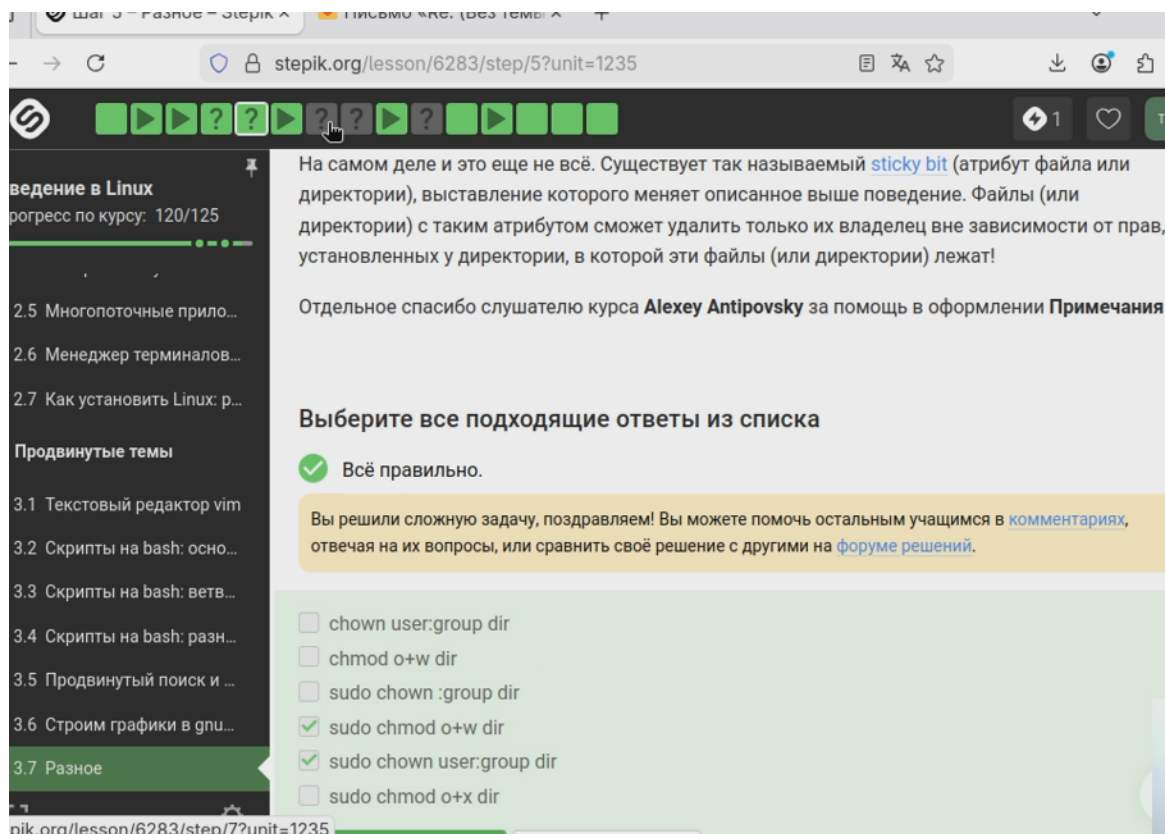


Рисунок 3.32

ws считает длину самой длинной строки (-L), размер в байтах (-c), количество строк (-l), но не считает буквы и предложения (рис. 3.33).

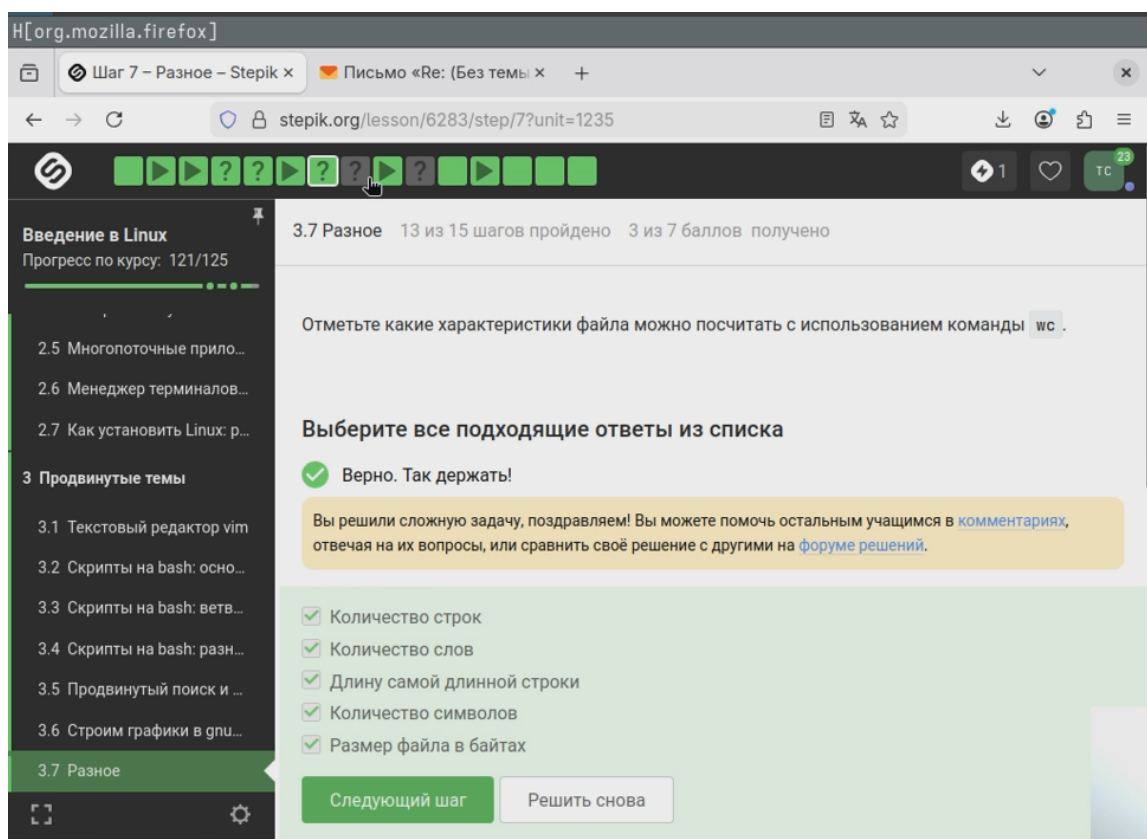


Рисунок 3.33

Команда `du -sh` (или `du -h -s`) выводит суммарный размер текущей директории в человеко-читаемом формате (например, 2.4M). Опция `-s` (summary) показывает только общий размер, без перечисления файлов (рис. 3.34).

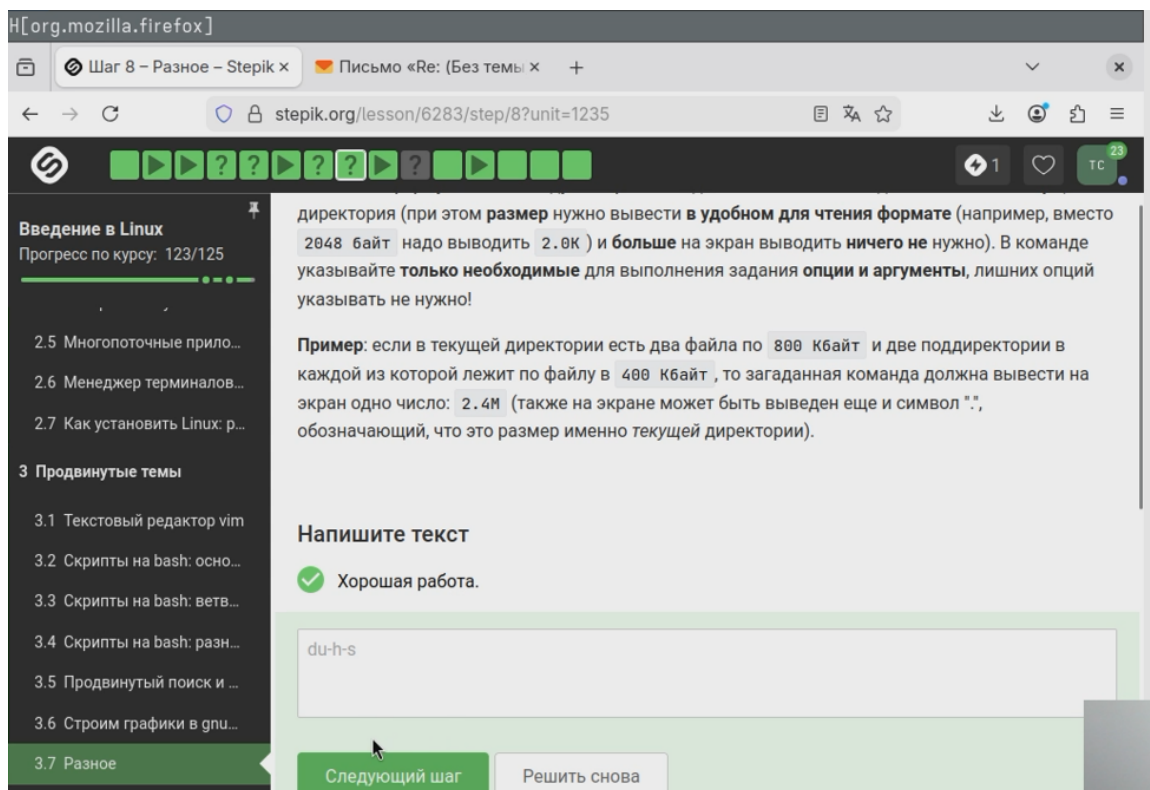


Рисунок 3.34

Самая короткая команда для создания трёх поддиректорий — `mkdir dir{1..3}`. Фигурные скобки в Bash генерируют последовательность: `dir1`, `dir2`, `dir3` (рис. 3.35).

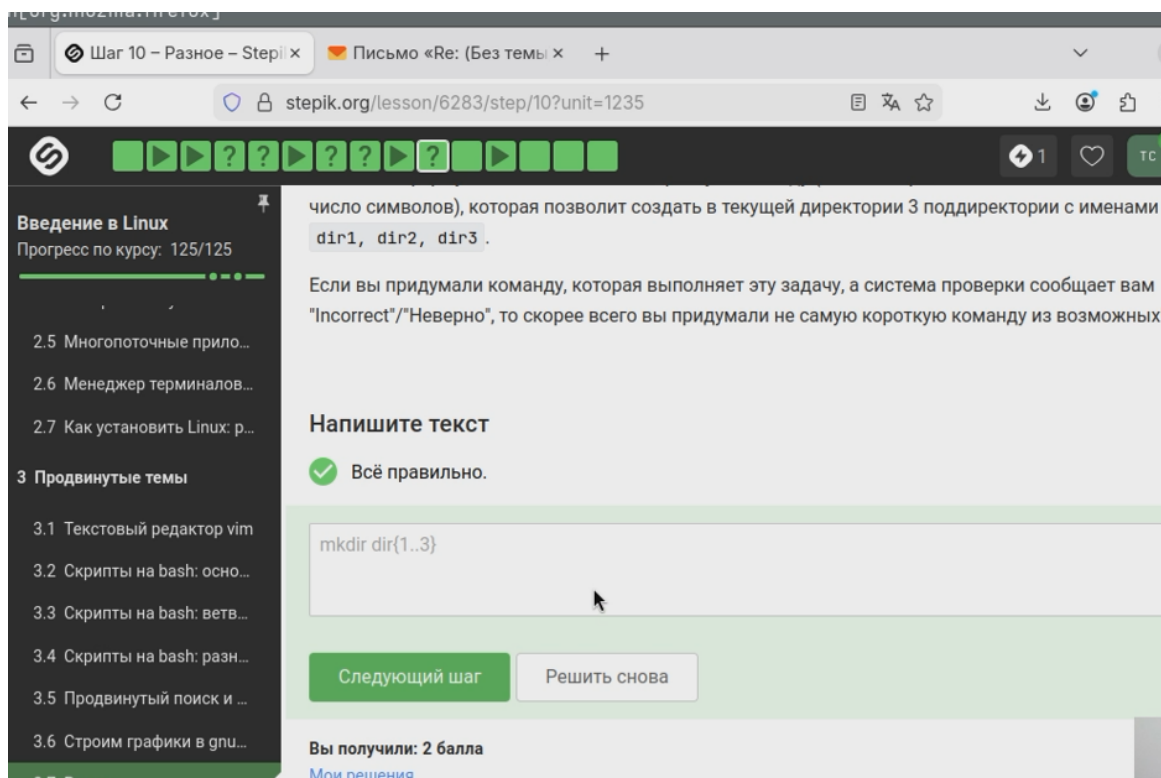


Рисунок 3.35

4 Выводы

Прошла Модуль 3 курса stepik «Введение в Linux!»